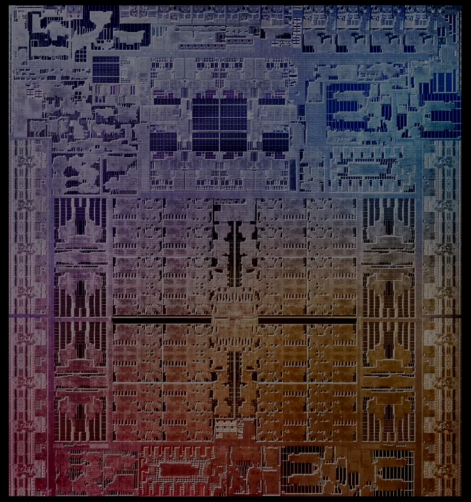
2026 반도체 설계 캠퍼스 시즌 2 (2026. 6. 30)

시스템 반도체 설계 기초 : CMOS 부터 EDA 까지

Seokhyeong Kang I POSTECH



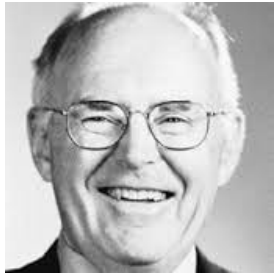
Apple, M1 Ultra('22)

TSMC 5nm FinFET (EUV)

114,000,000,000 *transistors on a single chip*

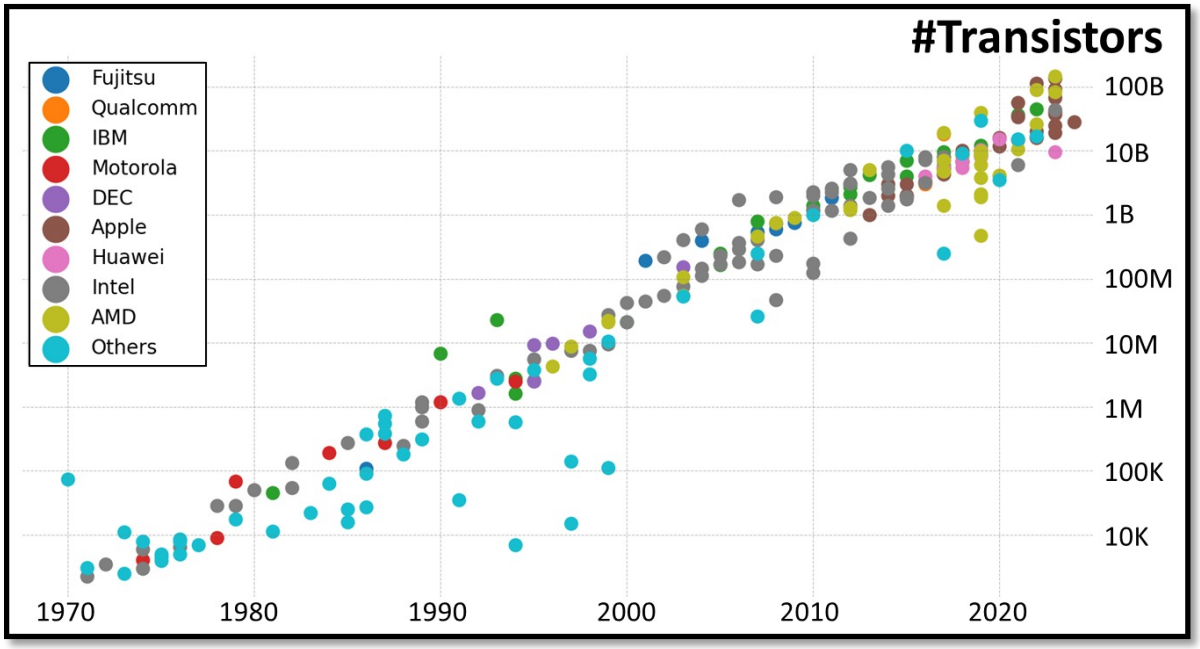
No one draws a billion switches by hand. So how is it even possible?

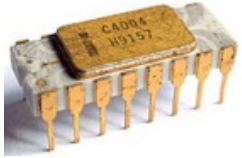
Moore's Law since 1965



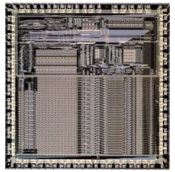
Gordon Moore
(Intel co-founder)

~ 2X Transistors for every two years.



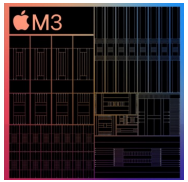


Jack Kilby's First IC
(Texas Inst, 1958)



ARM Processor
(Acorn, 1985)

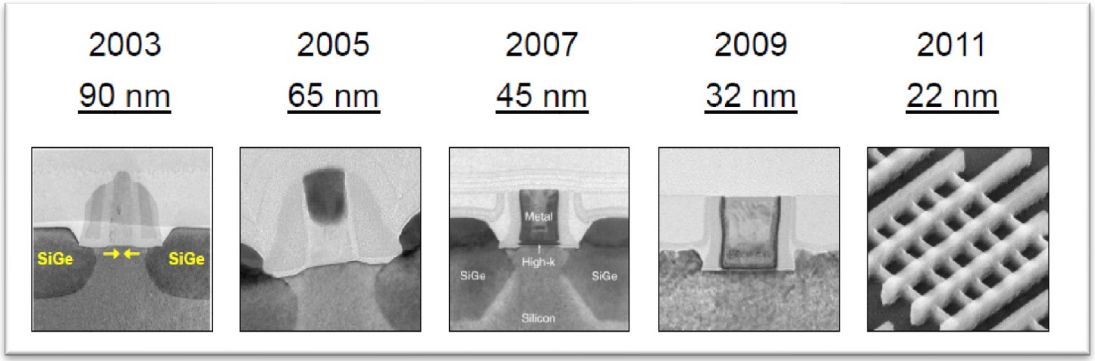
Intel i7
(Intel, 2008)

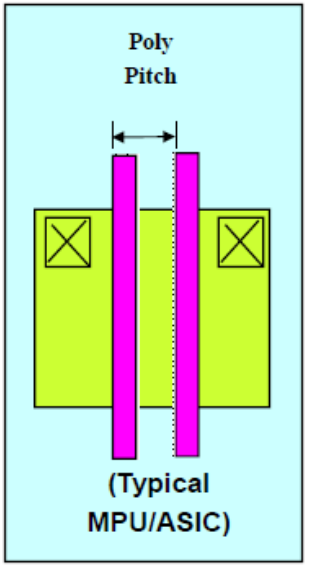


M3
(Apple, 2025)

Key Knobs for the Evolution: MOS Transistor Scaling

- S = 0.7 (0.5x per 2 nodes)
  - e.g., 130nm → 90nm → 65nm → 45nm → 32nm → 22nm → 16/14nm → 10nm → 8nm …



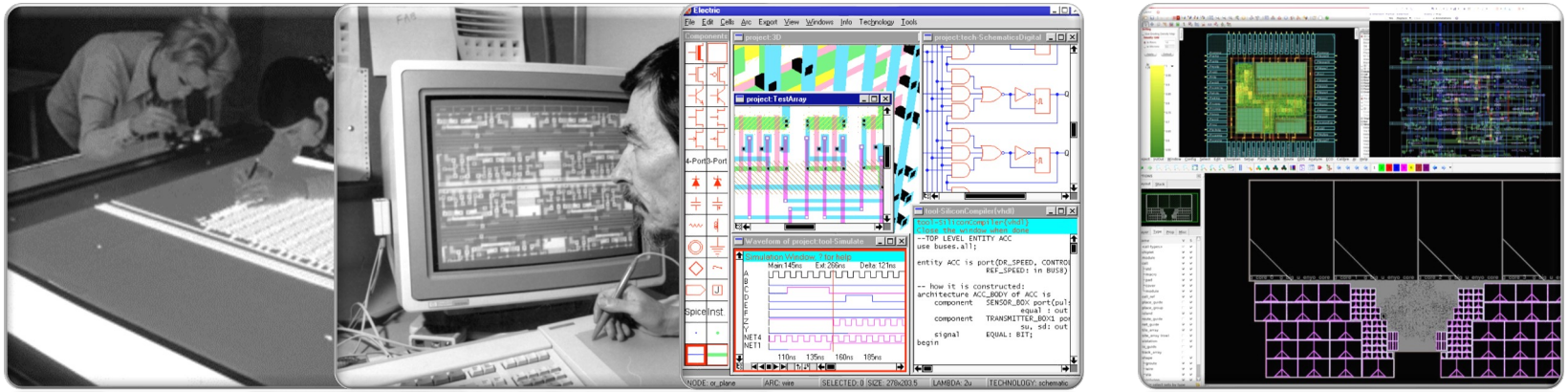


- Technology shrinks by 0.7/generation
- With every generation can integrate 2x more functions per chip
- Chip cost does not increase significantly; Cost of a function decreases by 2x

Key Knobs for the Evolution: Computer-Aided Design

- Electronic design automation (EDA) is a category of SW tools for designing electronic systems such as integrated circuits.
- Billions of components in modern semiconductor chip → EDA tools are essential.

1970s 1980 1990 2020s (Now)



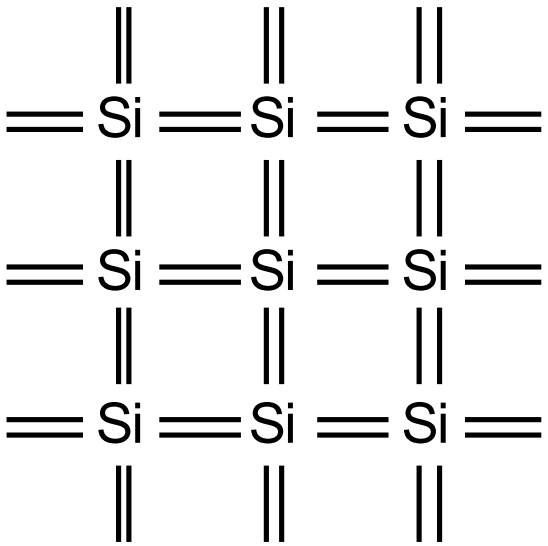
Contents

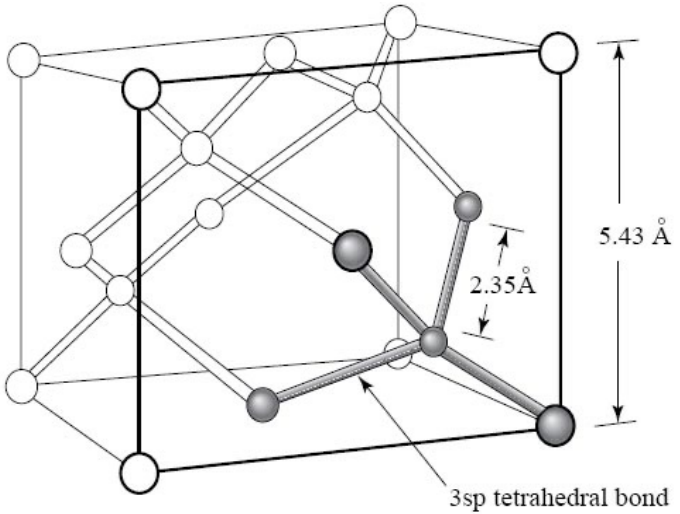
- CMOS Technology
- Timing and Power
- Design Methodology
- Electronic Design Automation

CMOS Technology

Silicon Semiconductors

- Modern electronic chips are built mostly on silicon substrates
- Silicon is a Group IV semiconducting material
- Crystal lattice: covalent bonds hold each atom to four neighbours

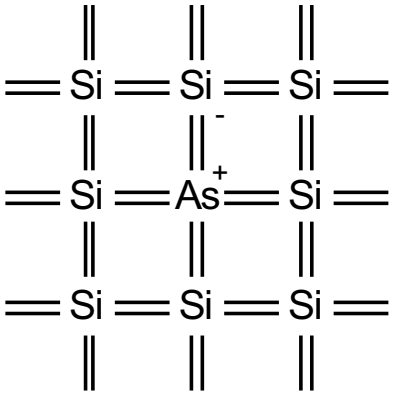


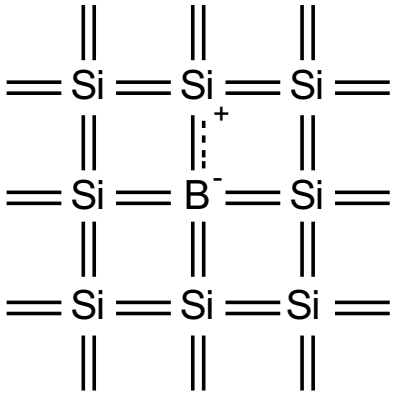


http://onlineheavytheory.net/silicon.html

Dopants

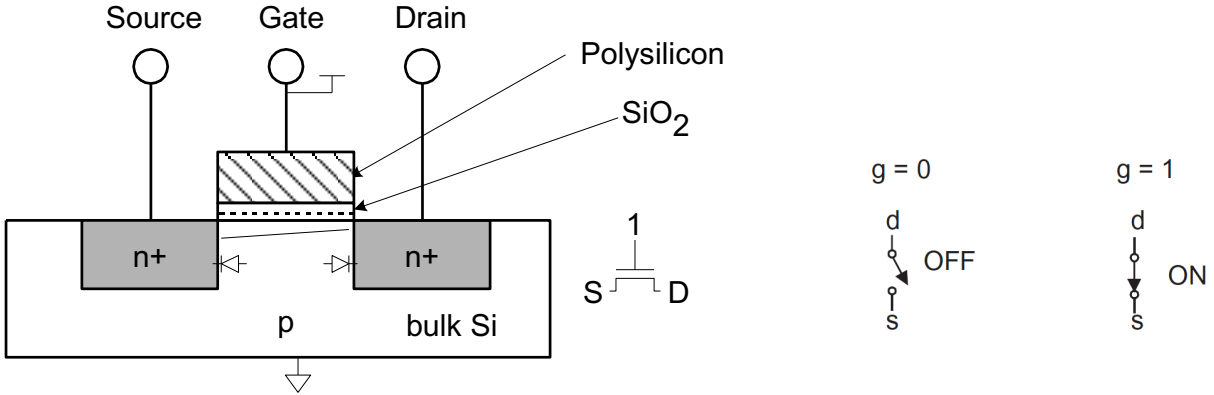
- Pure silicon has few free carriers and conducts poorly
- Adding dopants increases the conductivity drastically
- Dopant from Group V (e.g. As, P): extra electron (n-type)
- Dopant from Group III (e.g. B, Al): missing electron, called hole (p-type)





nMOS Operation

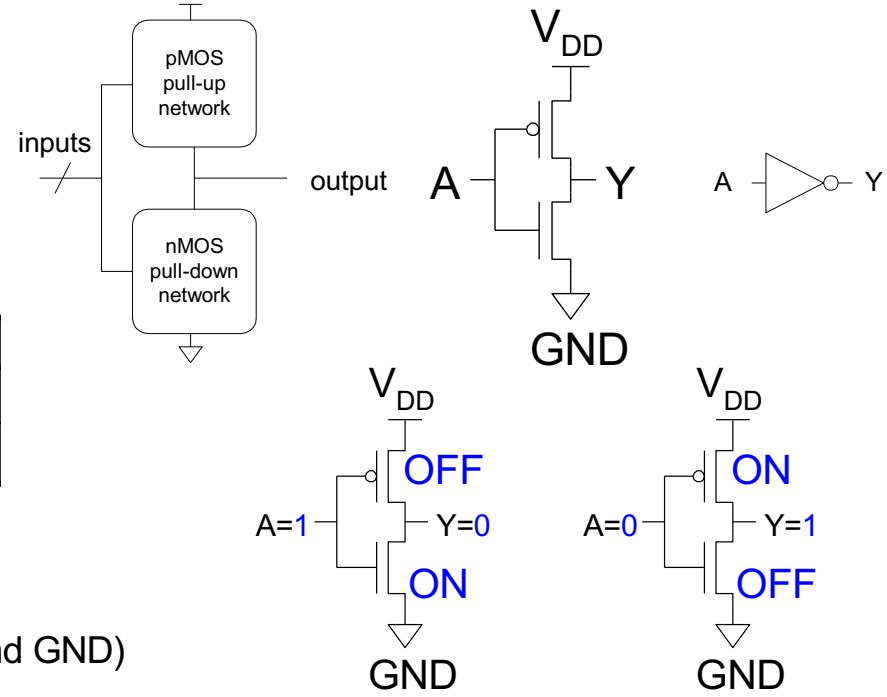
- When the gate is at a high voltage:
  - Positive charge on gate of MOS capacitor
  - Negative charge attracted to body
  - Inverts a channel under gate to n-type
  - Now current can flow through n-type silicon from source through channel to drain, transistor is ON



Complementary MOS

- Complementary MOS logic gates
  - nMOS *pull-down network*
  - pMOS *pull-up network*
  - a.k.a. static CMOS

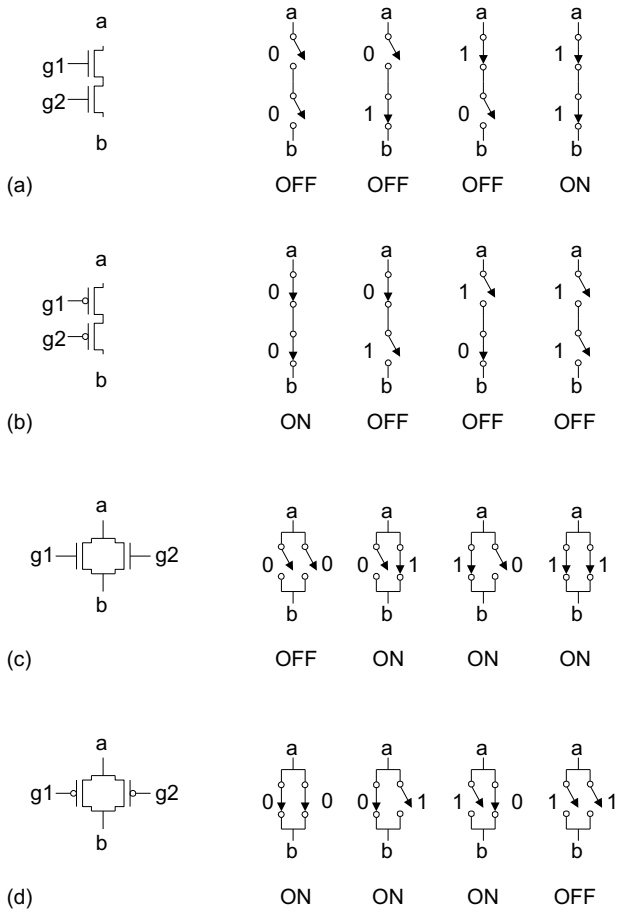
	Pull-up OFF	Pull-up ON
Pull-down OFF	Z (float)	1
Pull-down ON	0	X (crowbar)



- Advantage of CMOS
  - Very low static power (No DC path between VDD and GND)
  - Excellent noise margin (V_{OL} = 0, V_{OH} = VDD)
  - High operating speed

Series and Parallel

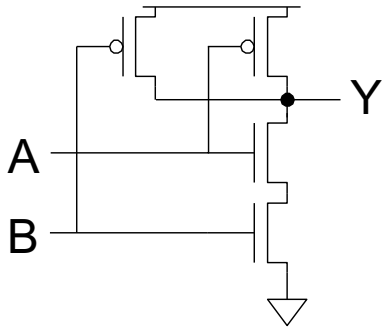
- nMOS: 1 = ON
- pMOS: 0 = ON
- *Series*: both must be ON
- *Parallel*: either can be ON



Conduction Complement

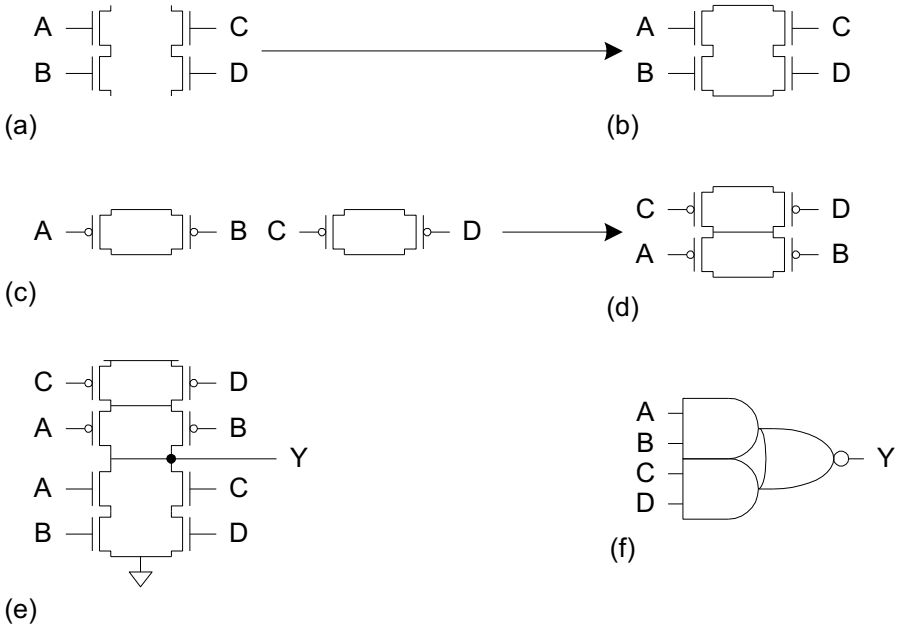
- CMOS gates always produce 0 or 1
- Ex: NAND gate
  - Series nMOS: Y=0 when both inputs are 1, Y=1 when either input is 0
  - Requires parallel pMOS

- Rule of Conduction Complements
  - Pull-up network is complement of pull-down
  - Parallel -> Series, Series -> Parallel



Compound Gates

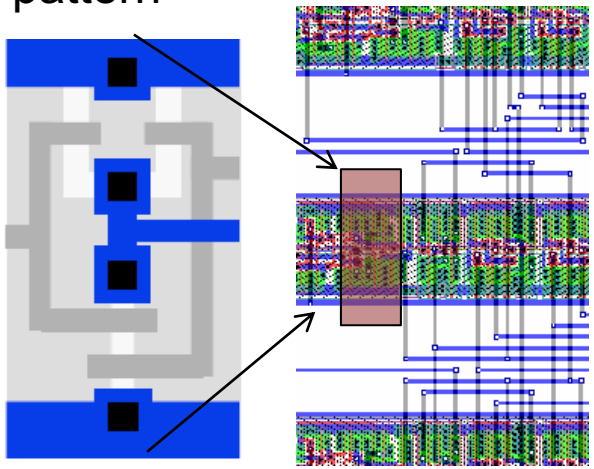
- *Compound gates* can do any inverting function
- Ex: $Y = \overline{A*B + C*D}$ (AND-OR-INVERT, AOI22)



Fabrication

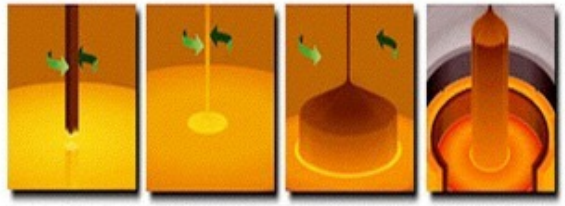
- Similar to photographic printing
  - Expose the silicon wafer through a mask
  - Process the silicon wafer
  - Repeat sequentially to pattern all the layers

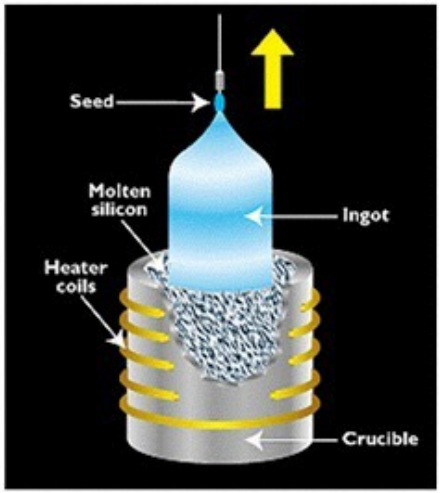
- Layout: A set of masks that tell a fabricator what to pattern
  - For each layer in your circuit
  - Layers are metal, drain/source implants, gate, etc.
  - You draw the layers; subject to vendor-supplied spacing rules



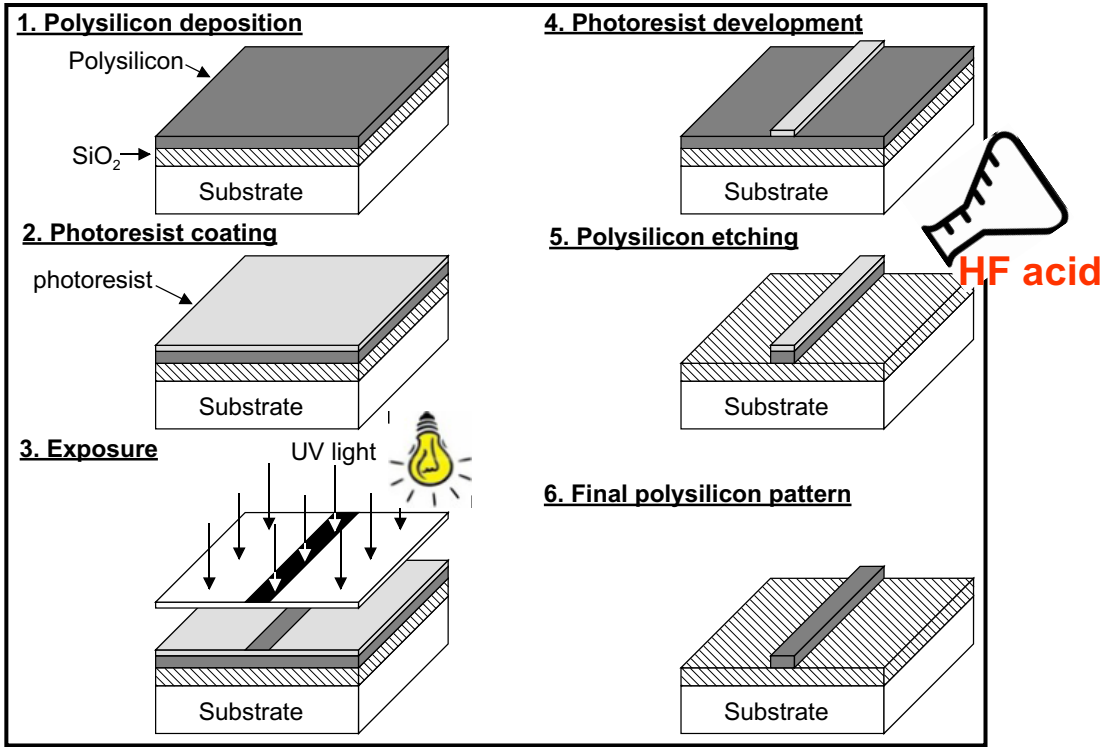
Growing the Silicon Ingot

- Silicon “ingots” are grown from a “perfect” crystal seed in a melt.
- They are then purified to “nine nines”.



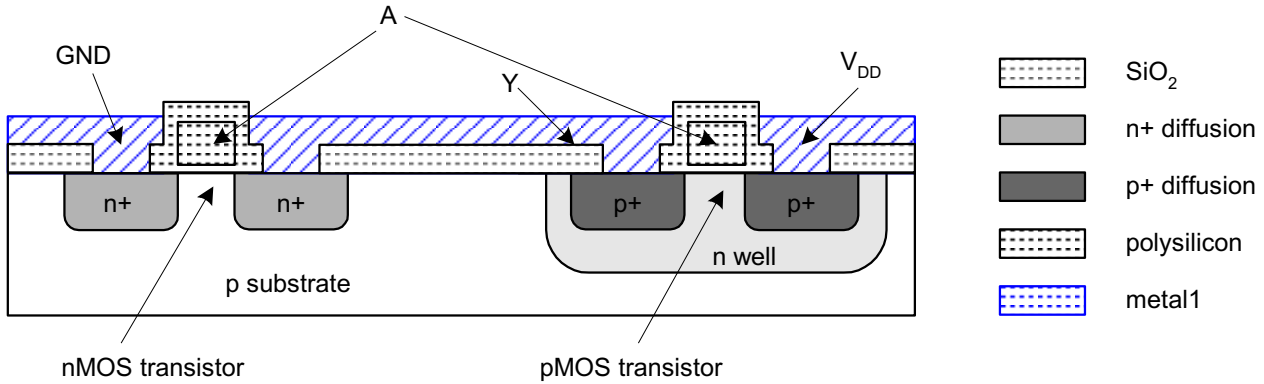


Patterning a layer above the silicon surface



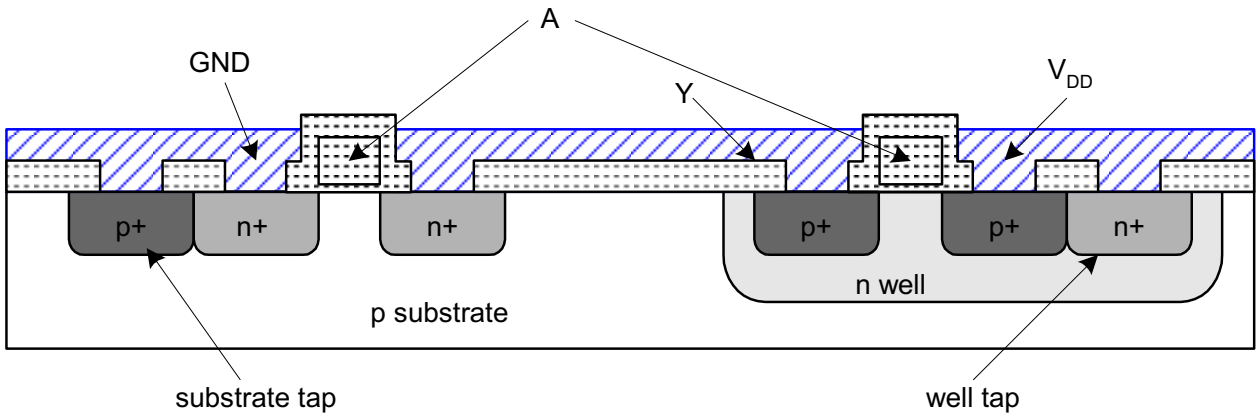
Inverter Cross-section

- Typically use p-type substrate for nMOS transistor
  - Requires n-well for body of pMOS transistors
  - Modern: Dual-Well Trench-Isolated CMOS Process

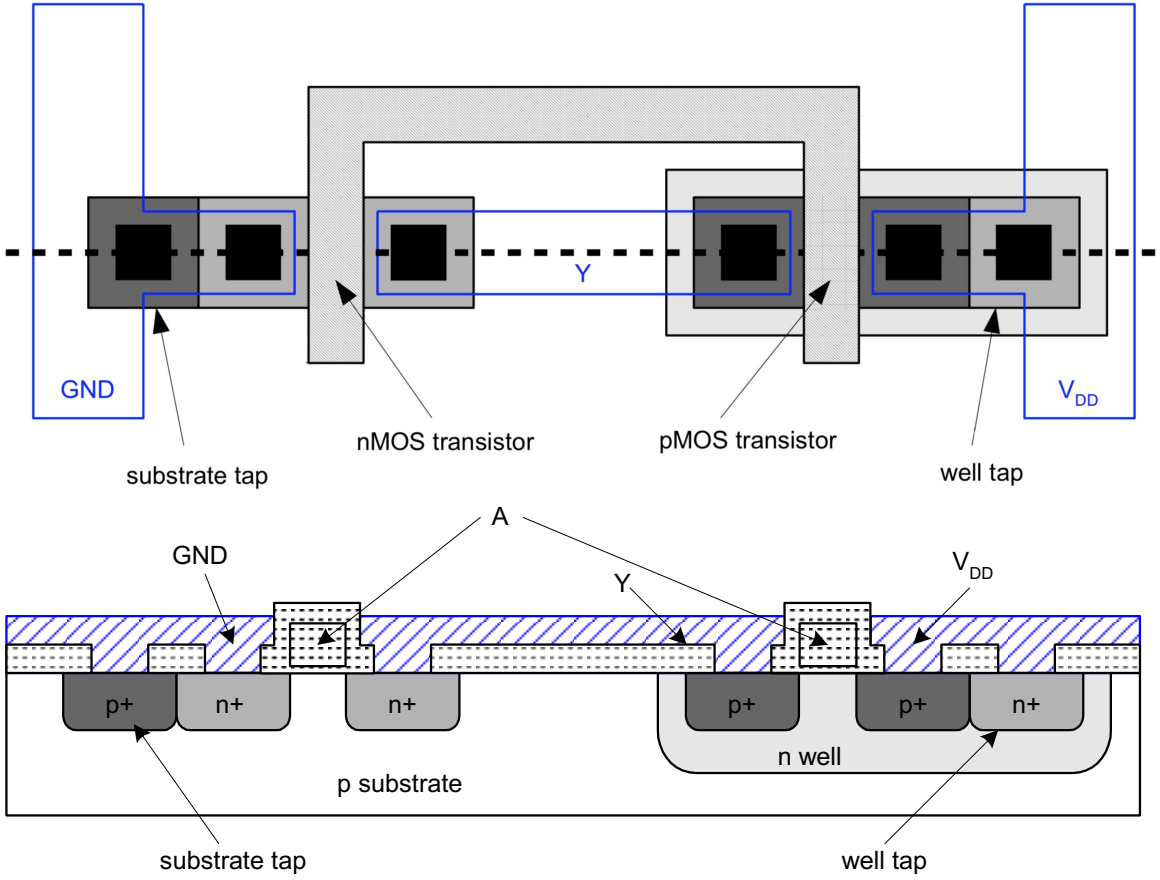


Well and Substrate Taps

- Substrate must be tied to GND and n-well to V_{DD}
- Metal to lightly-doped semiconductor forms poor connection called Shottky Diode
- Use heavily doped well and substrate contacts / taps

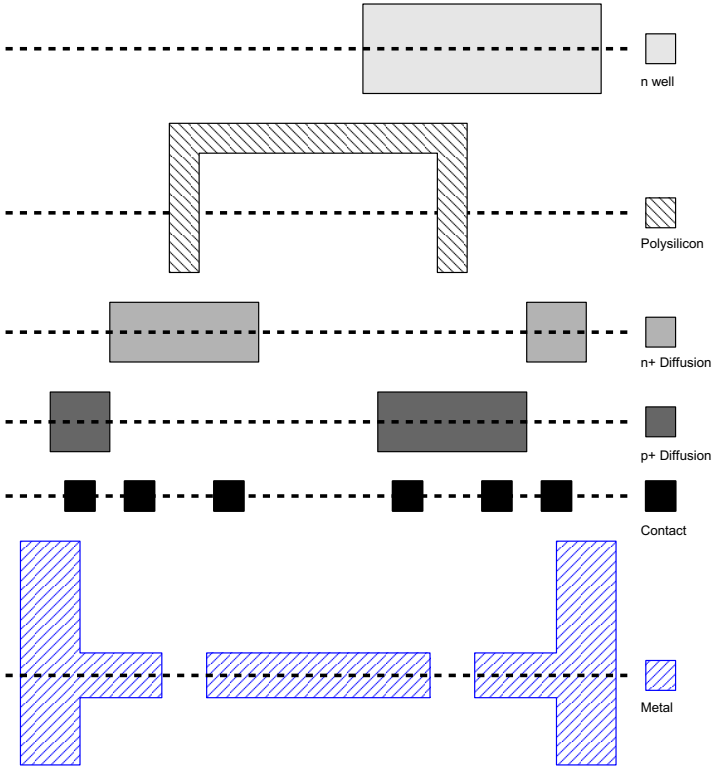


Each Layer Views



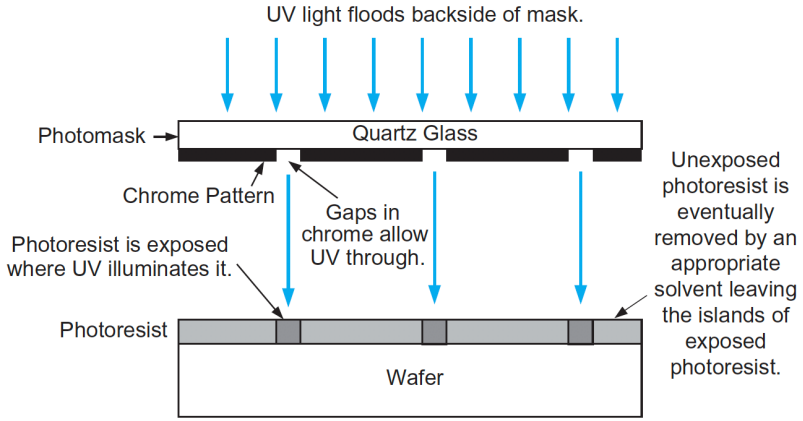
Detailed Mask Views

- Six masks
  - n-well
  - Polysilicon
  - n+ diffusion
  - p+ diffusion
  - Contact
  - Metal



Photolithography

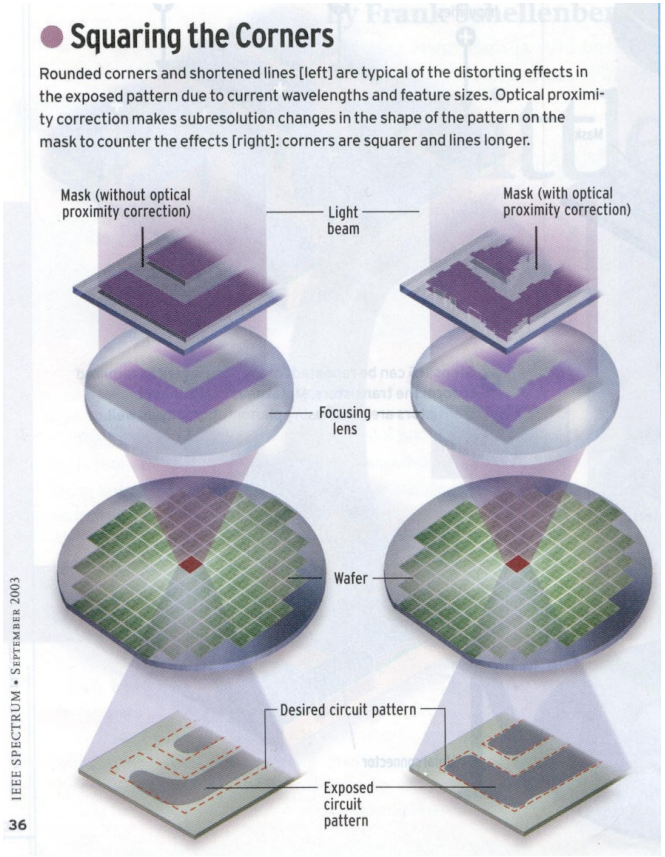
- Photolithography
  - IC manufacturing process which patterns a layout with an optical projection system



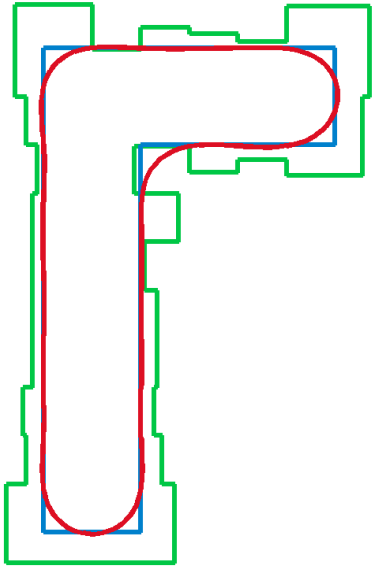
- Wave length of light source influences the minimum feature size
  - Minimum pitch (spacing + width) = $k \cdot \lambda / NA$

    (λ: wave length, NA: numerical aperture of lens)

OPC (Optical Proximity Correction)

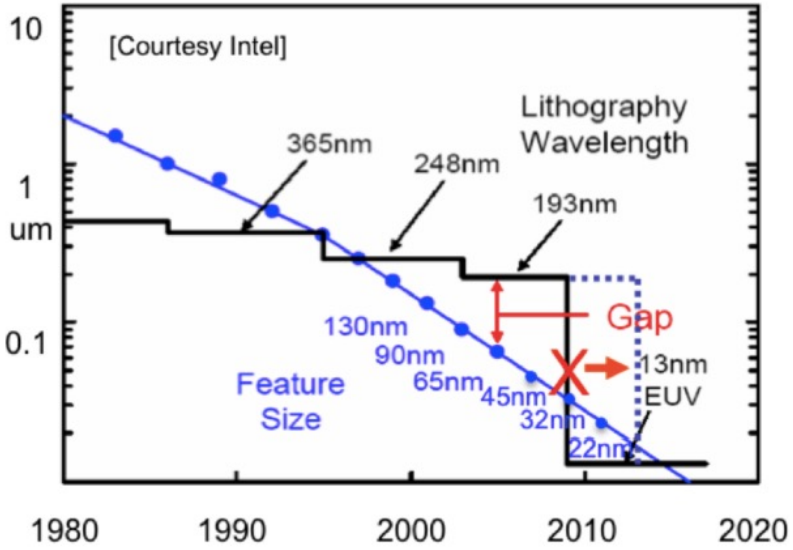


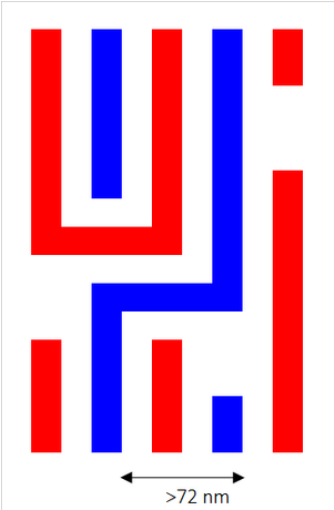
- OPC is a resolution enhancement technique
  - Due to light scattering, printed image blurs
    → OPC compensates the scattering / optical diffraction



Double Patterning

- Necessary before EUV (extreme ultraviolet) is introduced
  - Double patterning – assign adjacent features to two difference masks

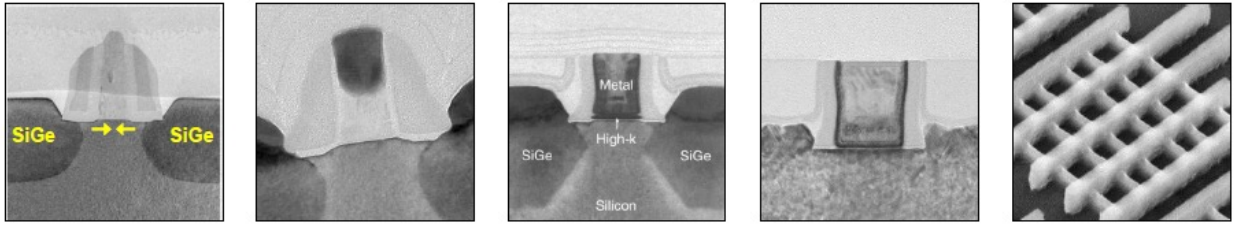




Transistor Technology (Intel)

[Mark Bohr, Intel]

2003	2005	2007	2009	2011
90 nm	65 nm	45 nm	32 nm	22 nm



Invented SiGe Strained Silicon	2nd Gen. SiGe Strained Silicon	Invented Gate-Last High-k Metal Gate	2nd Gen. Gate-Last High-k Metal Gate	First to Implement Tri-Gate

Strained Silicon

Better mobility

High-k Metal Gate

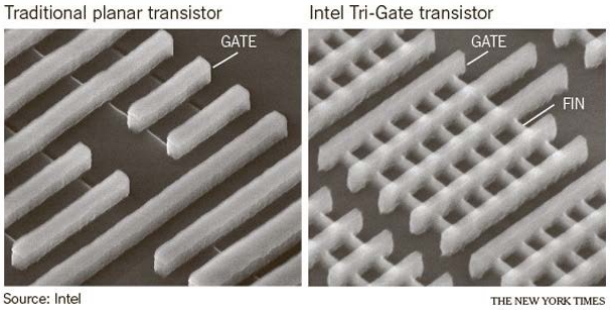
Increase gate cap.

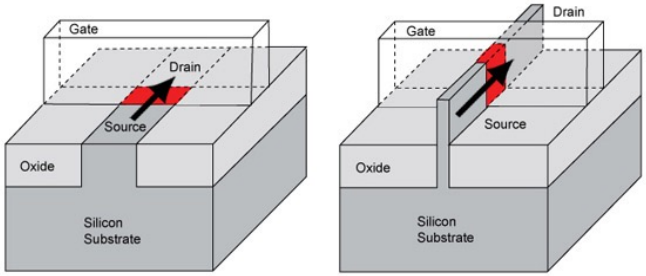
Tri-Gate

Increase on current
Reduce leakage

FinFET

- Conducting channel is wrapped by a thin silicon "fin", which forms the gate of the device.

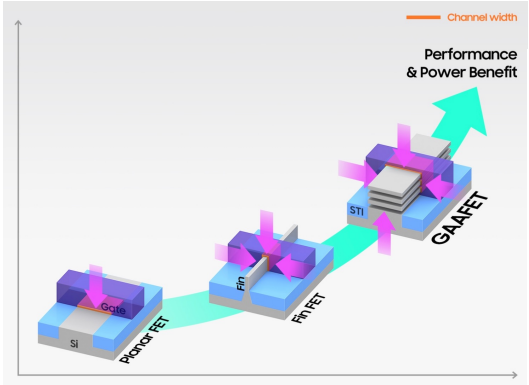


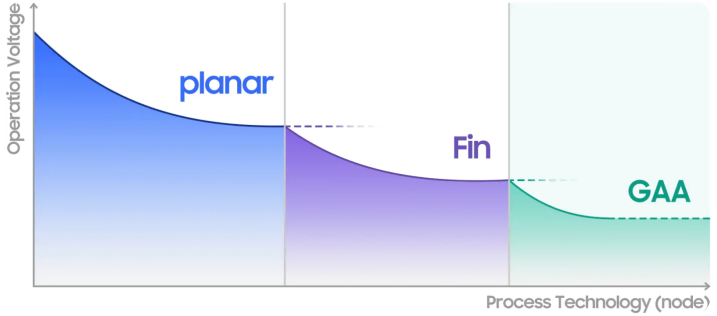


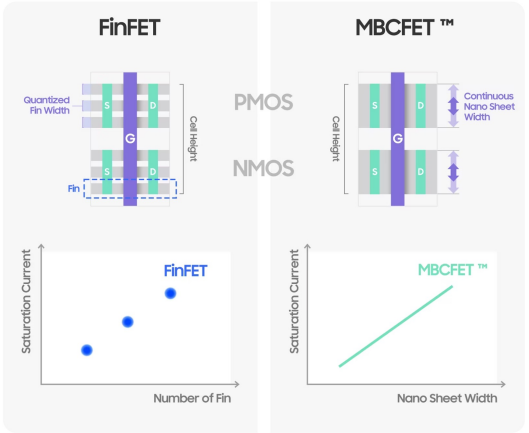
- Why FinFET?
  - Increase driving current → high performance
  - Better leakage current control → low power
  - Good for low voltage (power) applications
  - Compatible with current CMOS manufacturing

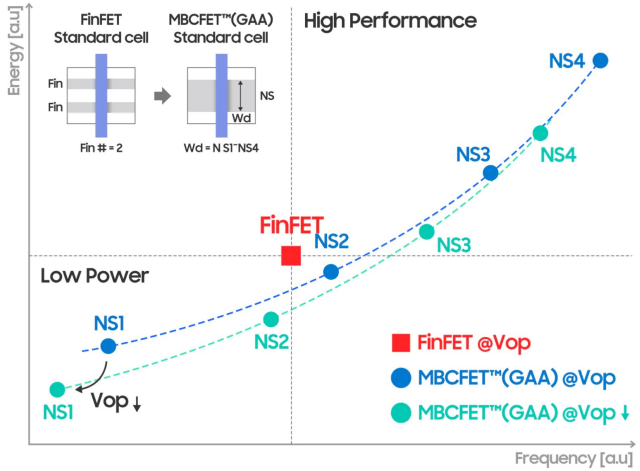
Samsung GAA MBCFET

[Samsung Tech Blog]



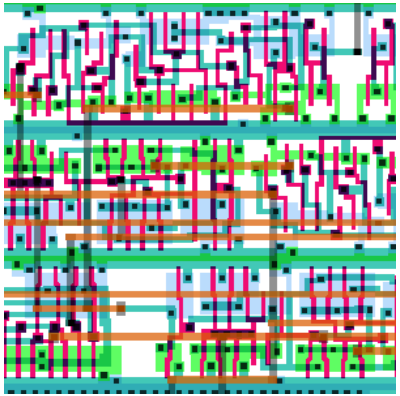






Layout

- Chips are specified with set of masks
- Minimum dimensions of masks determine transistor size (and hence speed, cost, and power)
- Feature size *f* = distance between source and drain
  - Set by minimum width of polysilicon
- Feature size improves 30% every tech. generation



Design Rules

- Interface between designer and process engineer
- Guidelines for constructing process masks

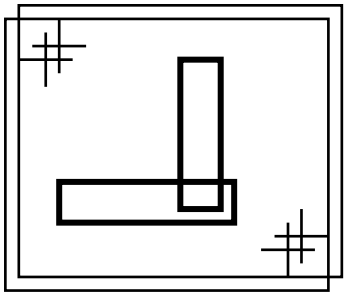
- Unit dimension: Minimum line width
  - scalable design rules: λ (lambda) parameter
  - absolute dimensions (micron rules)

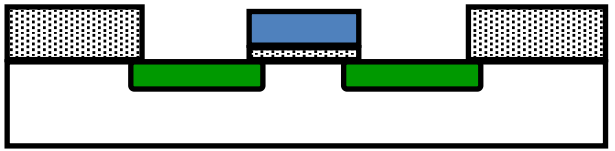
- Express rules in terms of $\lambda = f/2$

  (λ: parameter for design rule, f: minimum feature size)
  - E.g. λ = 0.3 μm in 0.6 μm process

Why Have Design Rules?

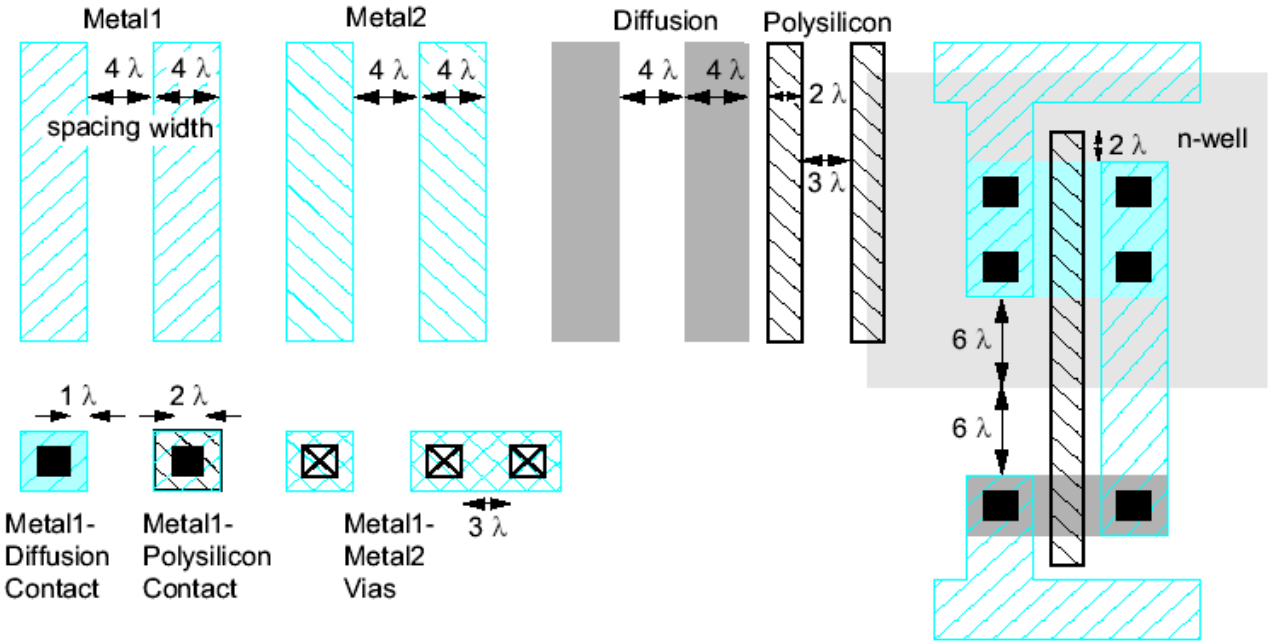
- To be able to tolerate some level of fabrication errors such as
  - 1. Mask misalignment
  - 2. Dust
  - 3. Process parameters (e.g., lateral diffusion)
  - 4. Rough surfaces





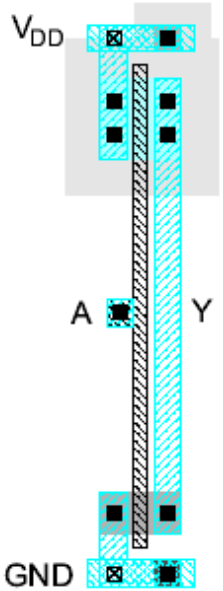
Simplified Design Rules

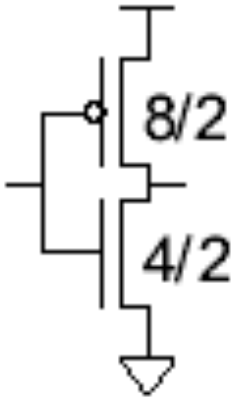
- Conservative rules to get you started

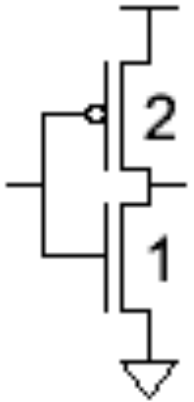


Inverter Layout

- Transistor dimensions specified as Width / Length
  - Minimum size is 4λ / 2λ, sometimes called 1 unit
  - In *f* = 0.6 μm process, this is 1.2 μm wide, 0.6 μm long

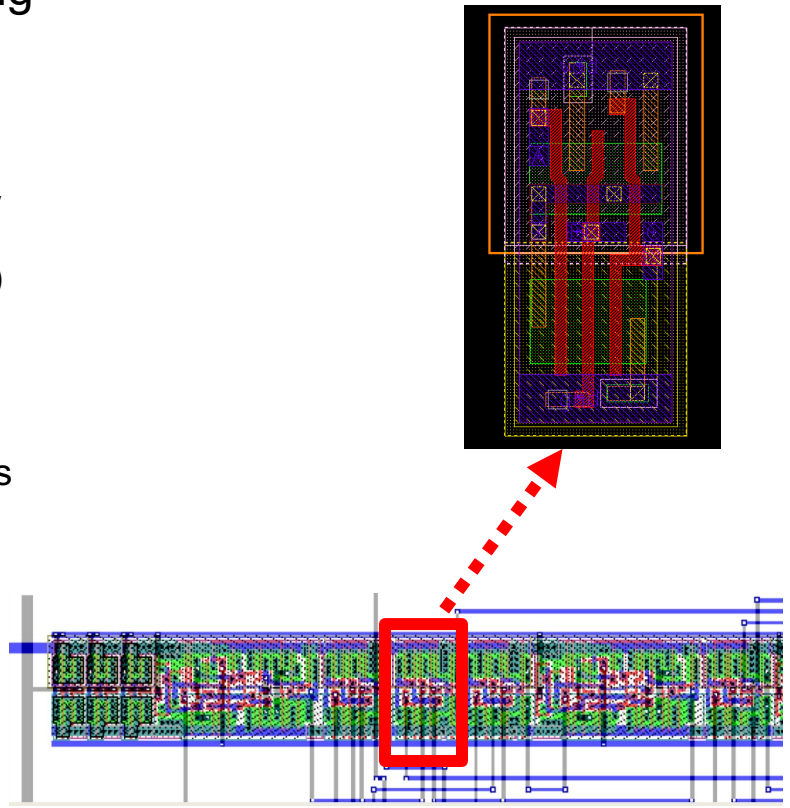


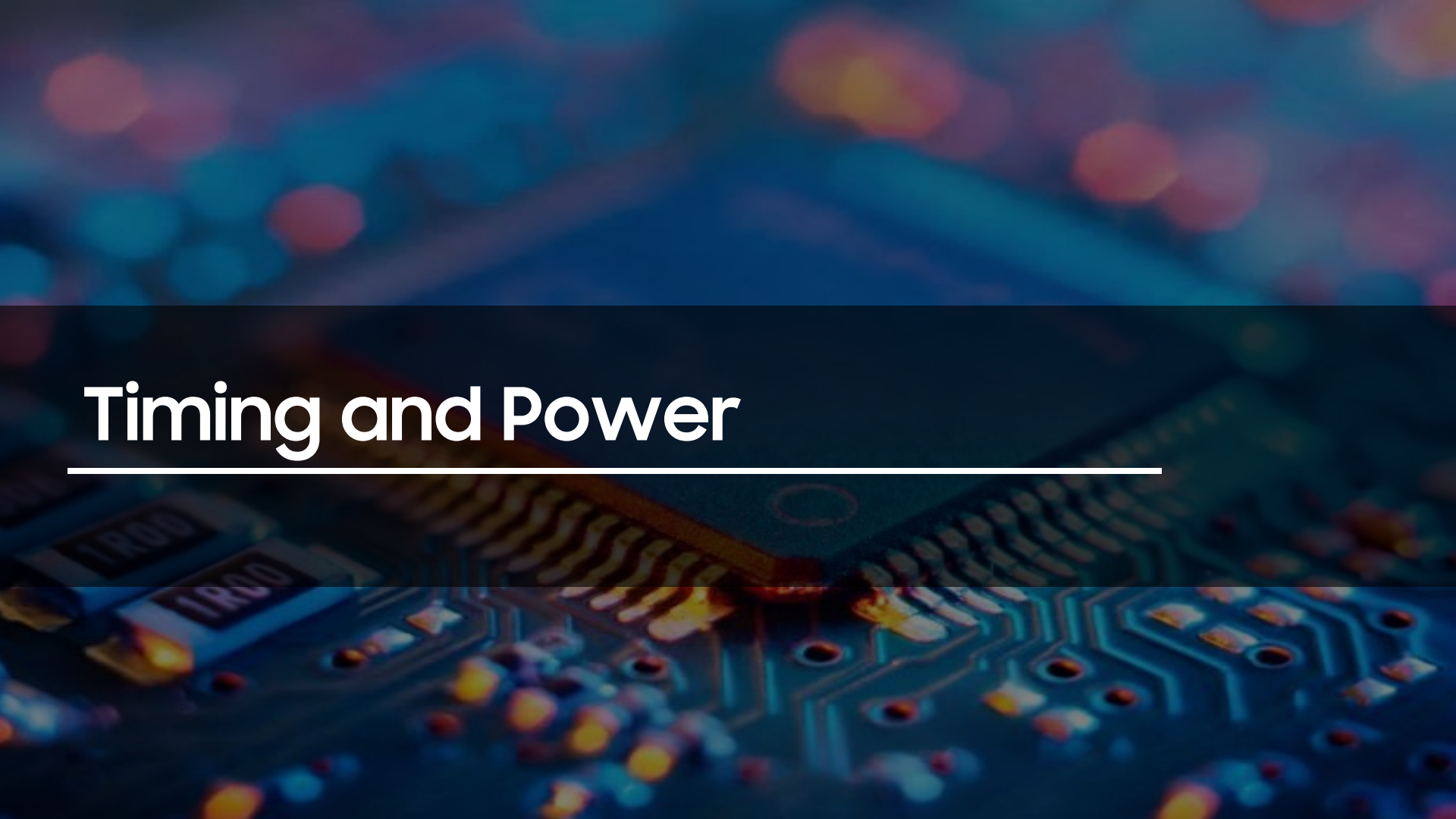




Gate Layout

- Layout can be very time consuming
  - Design gates to fit together nicely
  - Build a library of standard cells
- Standard cell design methodology
  - V_{DD} and GND should abut (standard height)
  - Adjacent gates should satisfy design rules
  - nMOS at bottom and pMOS at top
  - All gates include well and substrate contacts

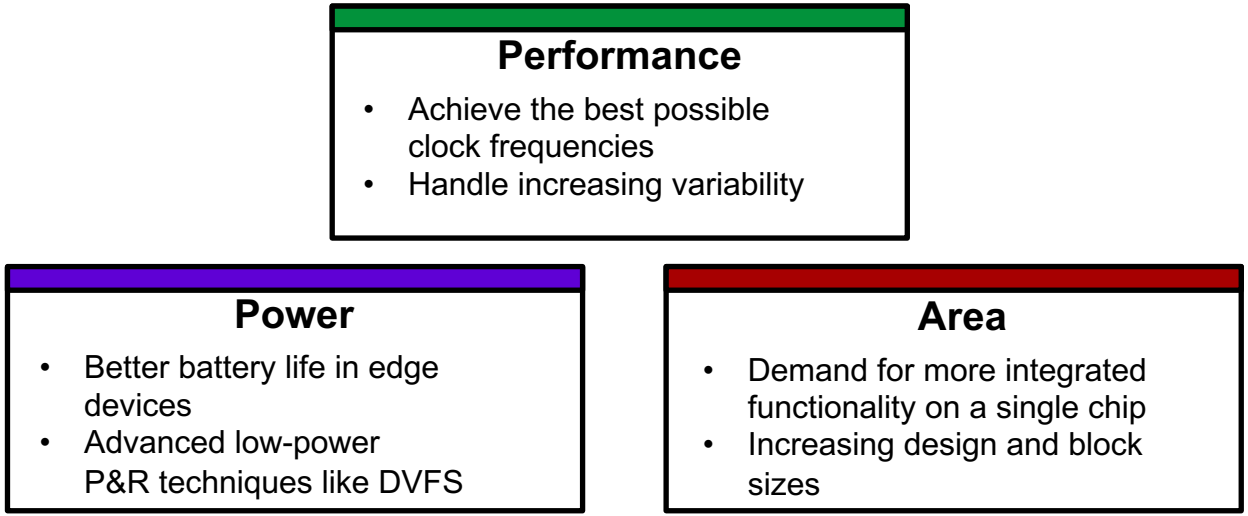




Timing and Power

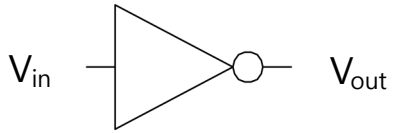
Design Objective

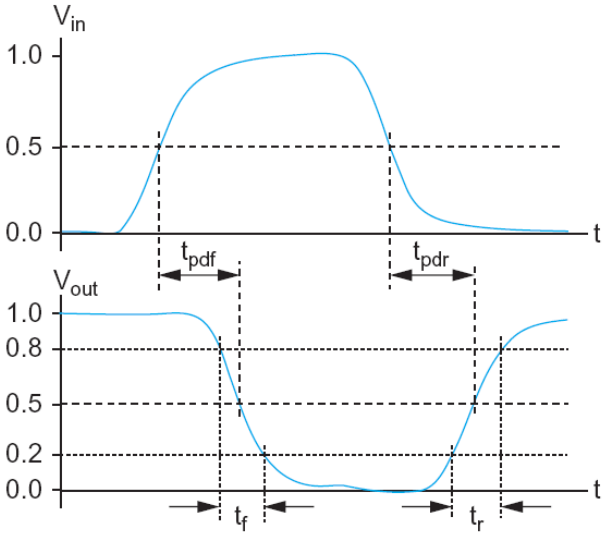
- **How to evaluate performance of a digital circuit?**
  - PPA (Power, Performance, Area)
  - + reliability, yield, cost …



Delay Definitions

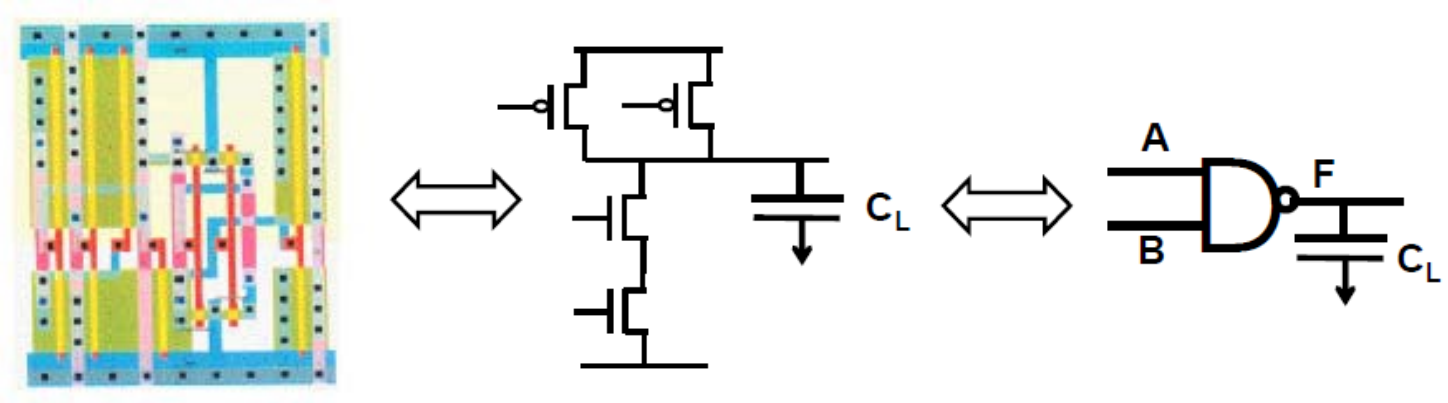
- t_{pdr}: *rising propagation delay*
  - From input to rising output crossing $V_{DD}/2$
- t_{pdf}: *falling propagation delay*
  - From input to falling output crossing $V_{DD}/2$
- t_{pd}: *average propagation delay*
  - $t_{pd} = (t_{pdr} + t_{pdf})/2$
- t_r: *rise time*
  - From output crossing 0.2 V_{DD} to 0.8 V_{DD}
- t_f: *fall time*
  - From output crossing 0.8 V_{DD} to 0.2 V_{DD}





Cell Delay in Real CAD Tool

Timing Characterization

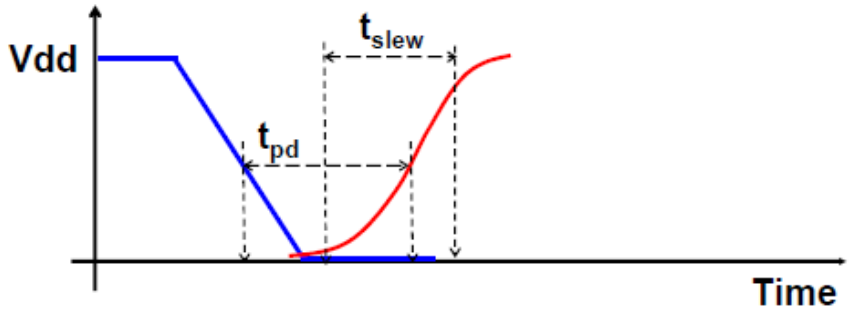


- "Extract" exact transistor characteristics from layout
  - Transistor width, length, junction area and perimeter
  - Local wire length and inter-wire distance
- Compute all transistor and wire capacitances

Cell Timing Characterization

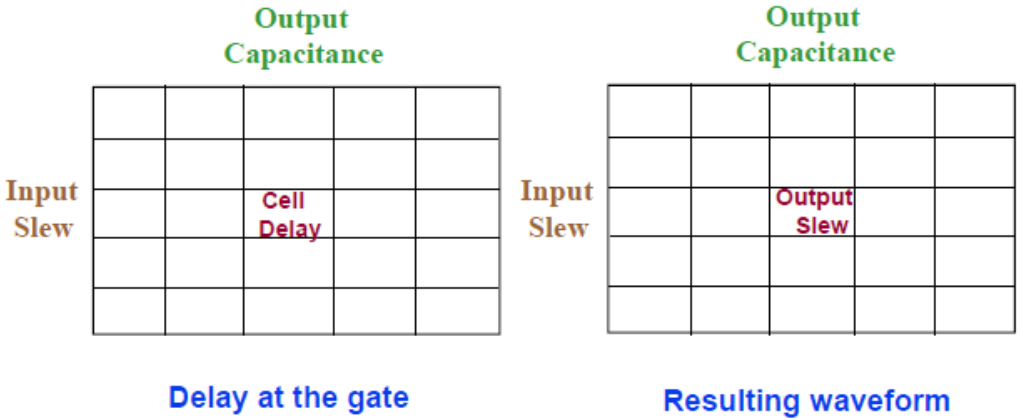
- Delay tables generated using a transistor-level circuit simulator such as HSPICE
  - Commercial tools: Encounter Library Characterizer (Cadence), SiliconSmart (Magma), etc

- For a number of different input slew and load capacitances simulate the circuit of the cell
  - Propagation time (50% Vdd at input to 50% at output)
  - Output slew (10% Vdd at output to 90% Vdd at output)



NLDM (nonlinear delay model)

- Nonlinear effects captured by lookup tables
- $D_G = f(C_L, S_{in})$
- $S_{out} = f(C_L, S_{in})$
- Timing tool interpolates between NLDM table entries
- Interpolation error is usually below 10% of SPICE



Timing Library Example (Synopsys Liberty, .lib)

```
cell("INV") {
pin(A) {
   max_transition : 1.500000;
   direction : input;
   rise_capacitance : 0.0739000;
   fall_capacitance : 0.0703340;
   capacitance : 0.07278646;
 }
 pin(Z) {
   direction : output;
   function : "!A";
   max_transition : 1.500000;
   max_capacitance : 5.1139;
   timing() {
    related_pin : "A";
    cell_rise(load) {
      index_1( "0.0375, 0.2329, 0.6904, 1.5008" );
      index_2( "0.0010, 0.9788, 2.2820, 5.1139" );
     values ( \
       "0.013211, 0.071051, 0.297500, 0.642340", \
       "0.028657, 0.110849, 0.362620, 0.707070", \
       "0.053289, 0.165930, 0.496550, 0.860400", \
       "0.091041, 0.234440, 0.661840, 1.091700" );
    }
    cell_fall(load) {
     index_1( "0.0326, 0.1614, 0.5432, 1.5017" );
     index_2( "0.0010, 0.4249, 3.6538, 8.1881" );
     values ( \
       "0.009472, 0.072284, 0.317370, 0.688390", \
       "0.009992, 0.095862, 0.360530,  0.731610", \
       "0.009994, 0.126620, 0.477260,  0.867670", \
       "0.009996, 0.144150, 0.644140,  1.127700" );
    }
    fall_transition(load) {
      index_1( "0.0326, 0.1614, 0.4192, 1.5017" );
      index_2( "0.0010, 0.4249, 2.1491, 8.1881" );
      values ( \
        "0.011974, 0.071668, 0.317600, 1.189560", \
        "0.033212, 0.101182, 0.328540,  1.189562", \
        "0.059282, 0.155052, 0.389900, 1.202360", \
        "0.162830, 0.317380, 0.628160, 1.441260" );
     }
     rise_transition(load) {
      index_1( "0.0375, 0.1650, 0.5455, 1.5078" );
      index_2( "0.0010, 0.4449, 1.7753, 5.1139" );
      values ( \
        "0.016690, 0.115702, 0.418200, 1.189060", \
        "0.038256, 0.139336, 0.422960,  1.189081", \
        "0.076248, 0.213280, 0.491620, 1.203700", \
        "0.170992, 0.353120, 0.694740, 1.384760" );
     }
    }
```

4 timing table:
{rise, fall} * {cell delay, transition time}

2-D Interpolation for Library LUTs

- Let's suppose we want to evaluate table value for (x,y) where $x_2<x<x_3$ and $y_1<y<y_2$
- Then the relevant entries are f_{21}, f_{31}, f_{22}, and f_{32}
- Procedure:
  - Compute weights along each dimension
  - Perform a weighted sum

Lookup Table

	x_1	x_2	x_3	x_4
y_1	f_{11}	f_{21}	f_{31}	f_{41}
y_2	f_{12}	f_{22}	f_{32}	f_{42}
y_3	f_{13}	f_{23}	f_{33}	f_{43}

Weights:

$w_x=(x-x_2)/(x_3-x_2)$

$w_y=(y-y_1)/(y_2-y_1)$

Value:

$f(x,y) = (1-w_x)(1-w_y)f_{21} + w_x(1-w_y)f_{31} + (1-w_x)w_yf_{22} + w_xw_yf_{32}$

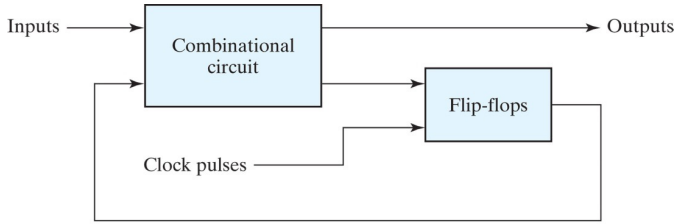
Example if $x=x_2$ and $y=y_2$:

→ $w_x=0$, $w_y=1$, $f(x,y) = 1*0*f_{21} + 0*0*f_{31} + 1*1*f_{22} + 0*1*f_{32} = f_{22}$

Sequential Logic

- Introduction
  - Sequential circuits act as storage elements and have memory, which can store, retain, and then retrieve information when needed at a later time
  - Output depends on current and previous inputs
  - Elements: latches and flip-flops

- Time sequence of sequential circuits

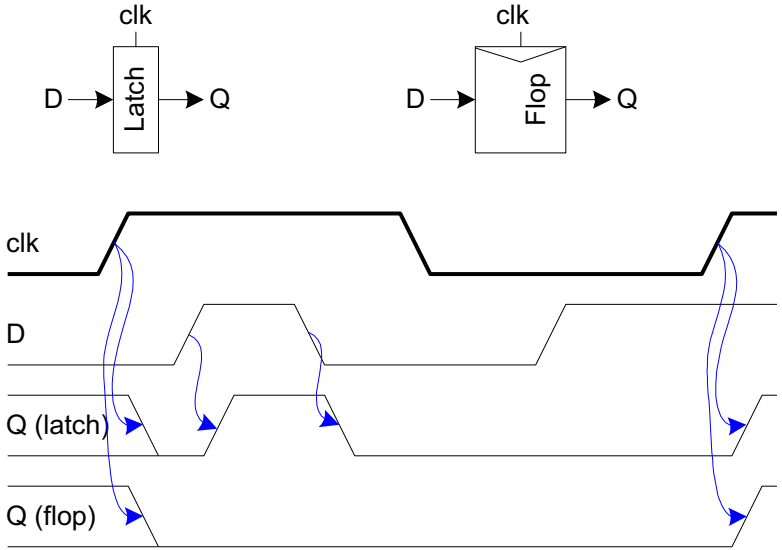


(a) Block diagram

(b) Timing diagram of clock pulses

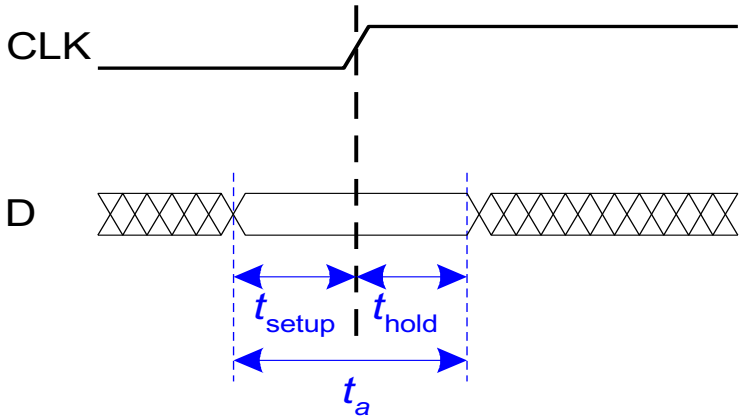
Sequencing Elements

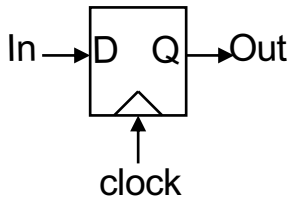
- Latch: Level sensitive
  - a.k.a. transparent latch, D latch
- Flip-flop: edge triggered
  - A.k.a. master-slave flip-flop, D flip-flop, D register
- Timing Diagrams
  - Transparent (latch)
  - Edge-trigger (flip-flop)



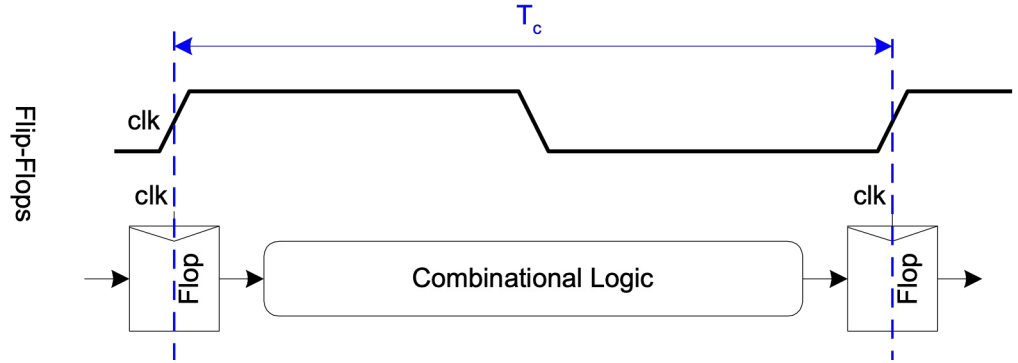
Input Timing Constraints

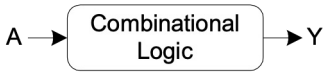
- Setup time: t_{setup}
  - time *before* the clock edge that data must be stable (i.e. not changing)
- Hold time: t_{hold}
  - time *after* the clock edge that data must be stable
- Aperture time: t_a
  - time around clock edge that data must be stable ($t_a = t_{setup} + t_{hold}$)

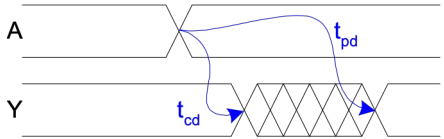


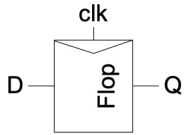


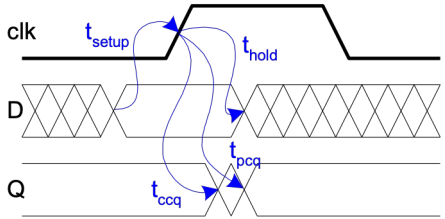
Timing Diagram





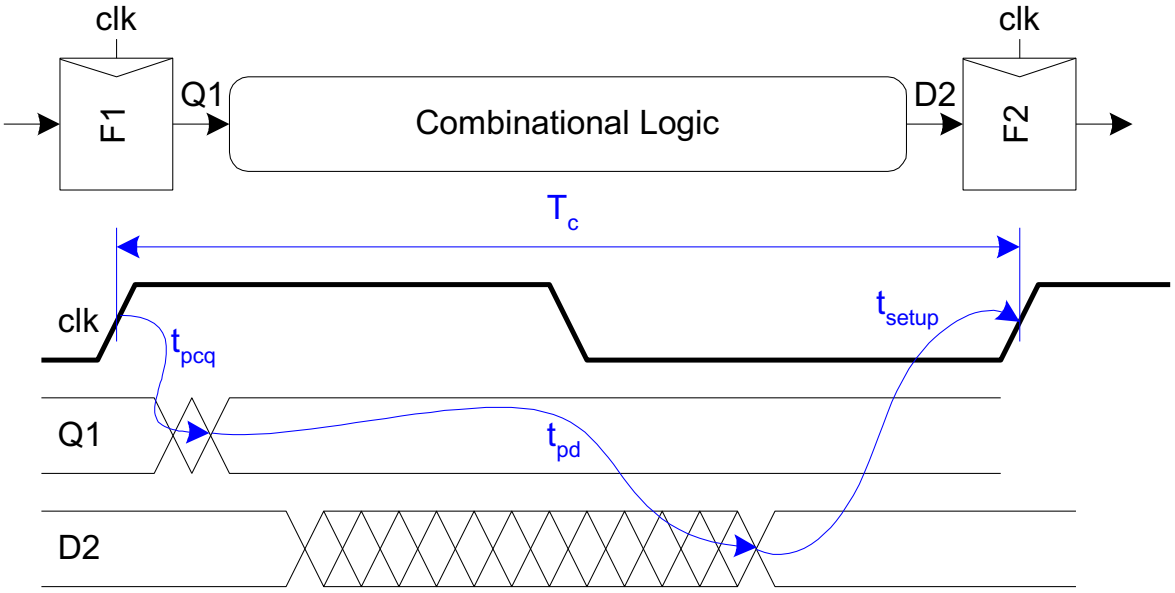






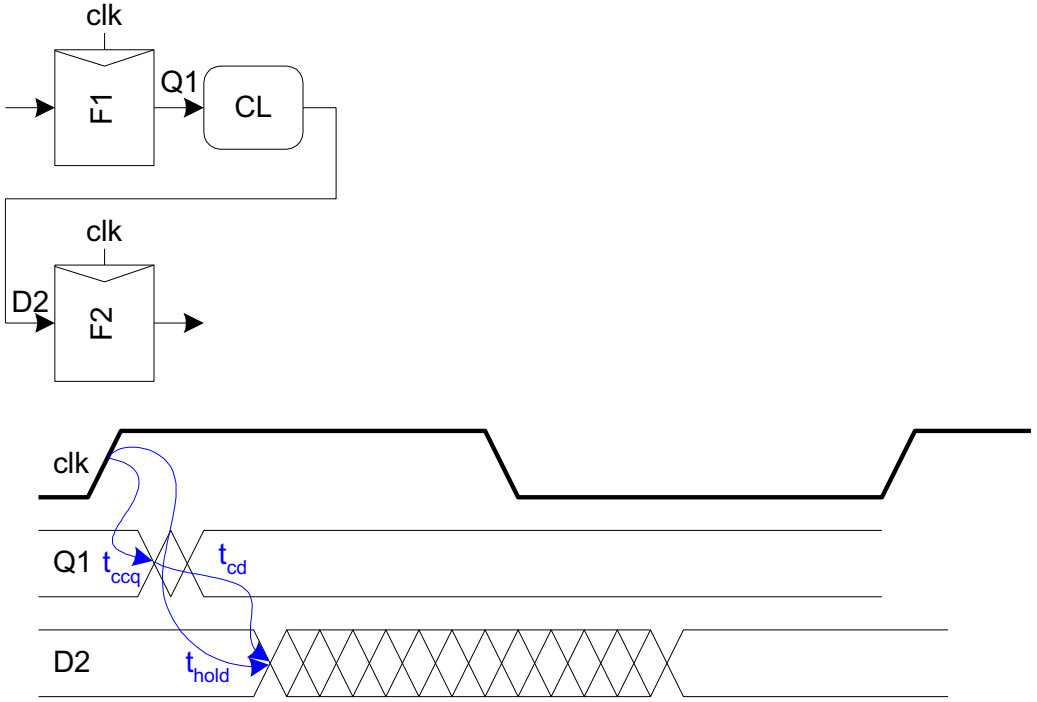
t_{pd}	Logic propagation delay
t_{cd}	Logic contamination delay
t_{pcq}	Clk->Q propagation delay
t_{ccq}	Clk->Q contamination delay
t_{setup}	Setup time
t_{hold}	Hold time

Max-Delay (Setup Time) Constraint



$$t_{pd} \leq T_c - (t_{setup} + t_{pcq})$$

Min-Delay (Hold Time) Constraint



$$t_{cd} \geq t_{\text{hold}} - t_{ccq}$$

Power Dissipation Sources

- $P_{total} = P_{dynamic} + P_{static}$
- Dynamic power: $P_{dynamic} = P_{switching} + P_{shortcircuit}$
  - Power consumption when chip is working
  - Switching load capacitances (1->0, 0->1)
  - Short-circuit current
- Static power: $P_{static} = (I_{sub} + I_{gate} + I_{junction})V_{DD}$
  - Power consumption when chip is in idle state
  - Subthreshold leakage
  - Gate leakage
  - Junction leakage

Power and Energy

- Power is drawn from a voltage source attached to the V_{DD} pin(s) of a chip.

- Instantaneous Power: $P(t) = I(t)V(t)$

- Energy: $E = \int_0^T P(t)dt$

- Average Power: $P_{\text{avg}} = \frac{E}{T} = \frac{1}{T}\int_0^T P(t)dt$

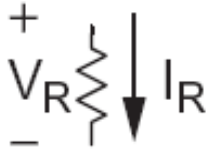
Power in Circuit Elements

Voltage source

$$P_{VDD}(t) = I_{DD}(t) V_{DD}$$

Resistor

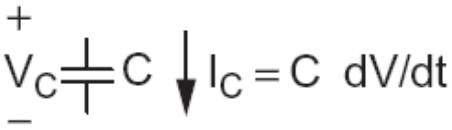
$$P_R(t) = \frac{V_R^2(t)}{R} = I_R^2(t) R$$



Capacitor

$$E_C = \int_0^\infty I(t) V(t)\, dt = \int_0^\infty C \frac{dV}{dt} V(t)\, dt$$

$$= C \int_0^{V_C} V(t)\, dV = \tfrac{1}{2} C V_C^2$$



Charging a Capacitor

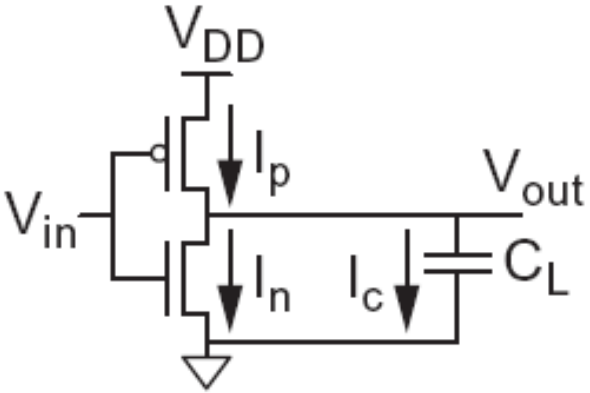
- When the gate output rises
  - Energy stored in capacitor is

$$E_C = \tfrac{1}{2} C_L V_{DD}^2$$

  - But energy drawn from the supply is

$$E_{VDD} = \int_0^\infty I(t) V_{DD} dt = \int_0^\infty C_L \frac{dV}{dt} V_{DD} dt$$

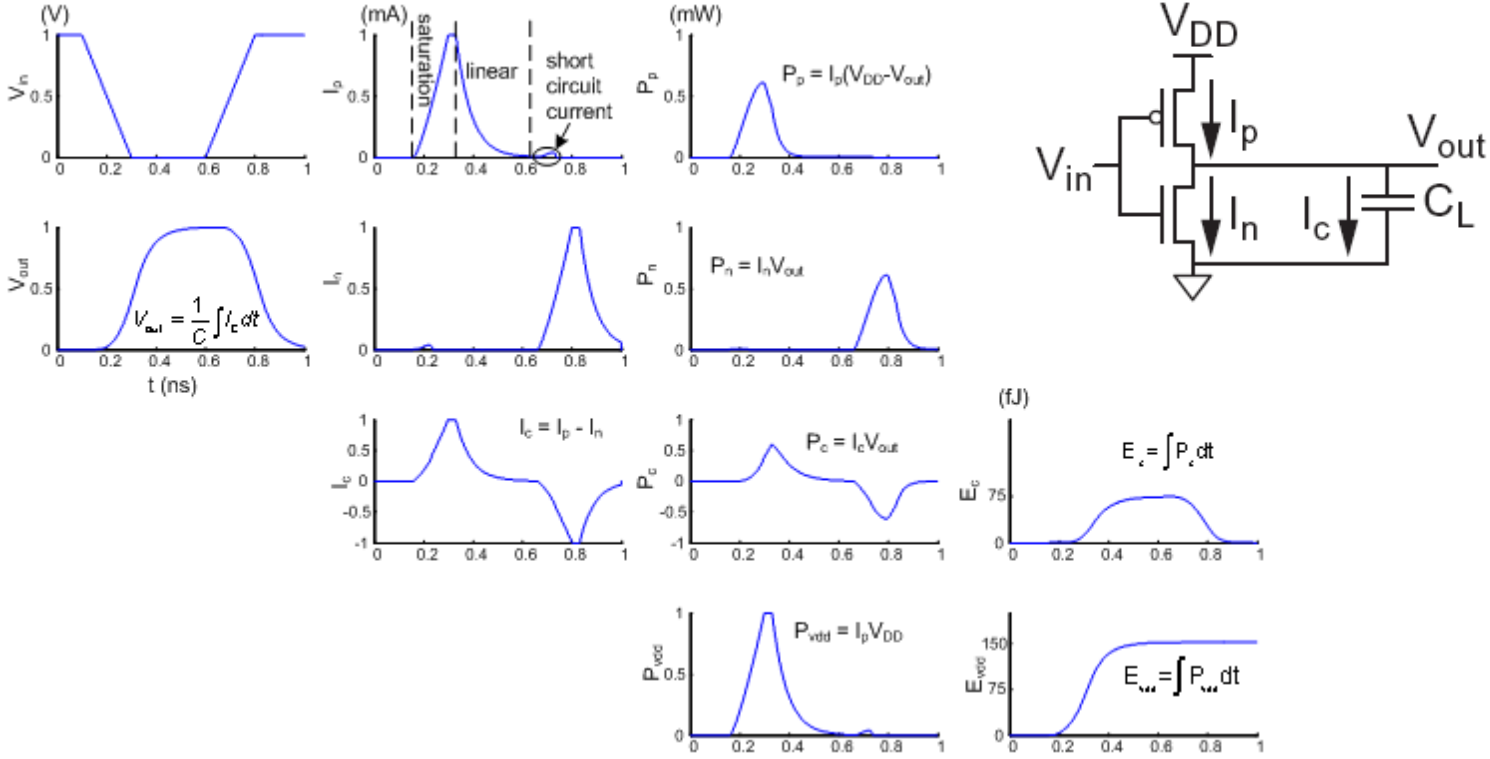
$$= C_L V_{DD} \int_0^{V_{DD}} dV = C_L V_{DD}^2$$



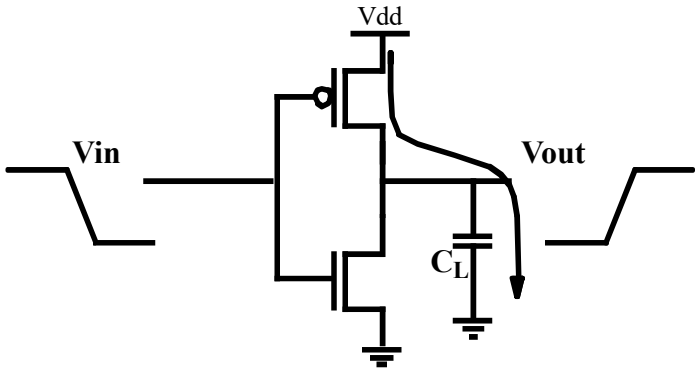
  - Half the energy from V_{DD} is dissipated in the pMOS transistor as heat, other half stored in capacitor
- When the gate output falls
  - Energy in capacitor is dumped to GND
  - Dissipated as heat in the nMOS transistor

Switching Waveforms

- Example: V_{DD} = 1.0 V, C_L = 150 fF, f = 1 GHz



Switching Power Dissipation



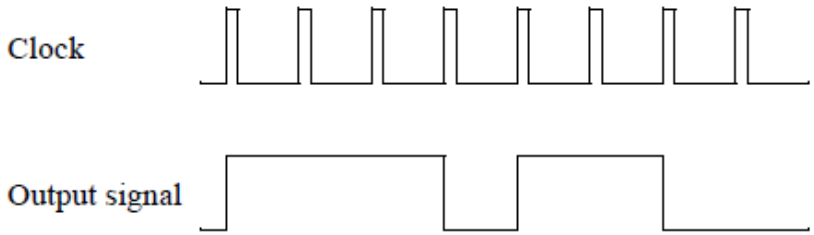
$$\text{Energy/transition} = C_L * V_{dd}^2$$

$$\text{Power} = \text{Energy/transition} * f = C_L * V_{dd}^2 * f$$

- Not a function of transistor sizes!
- Need to reduce C_L, V_{dd}, and *f (frequency)* to reduce power

Activity Factor

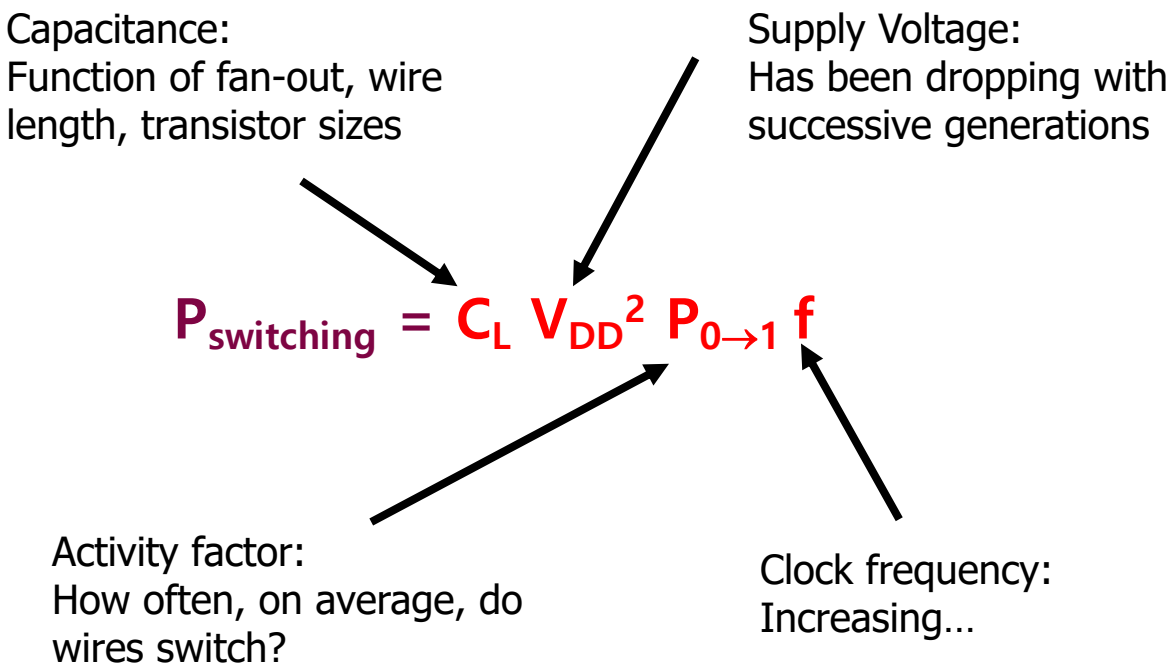
- Suppose the system clock frequency = f
- Let $f_{sw} = \alpha f$, where α = activity factor
  - If the signal is a clock, $\alpha = 1$
  - If the signal switches once per cycle, $\alpha = ½$



$\alpha = 0.25$

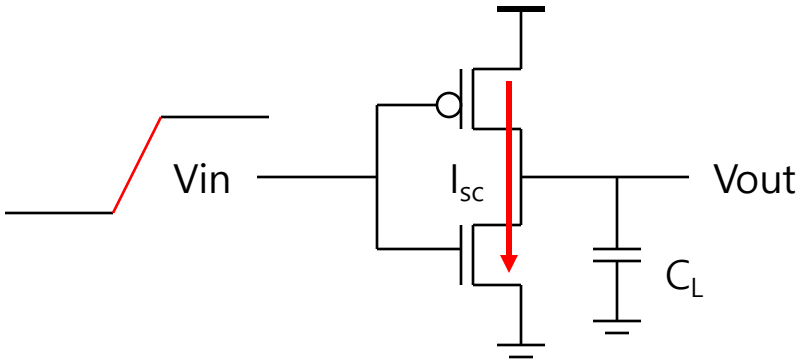
- Switching power: $P_{\text{switching}} = \alpha C V_{DD}^{\ 2} f$

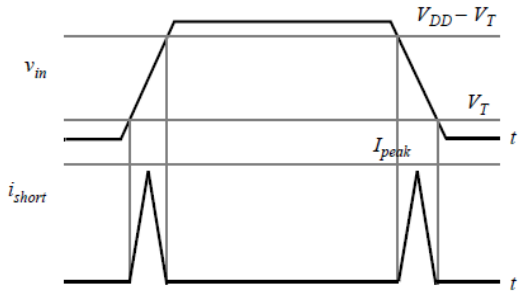
Lowering Switching Power



Short Circuit Current

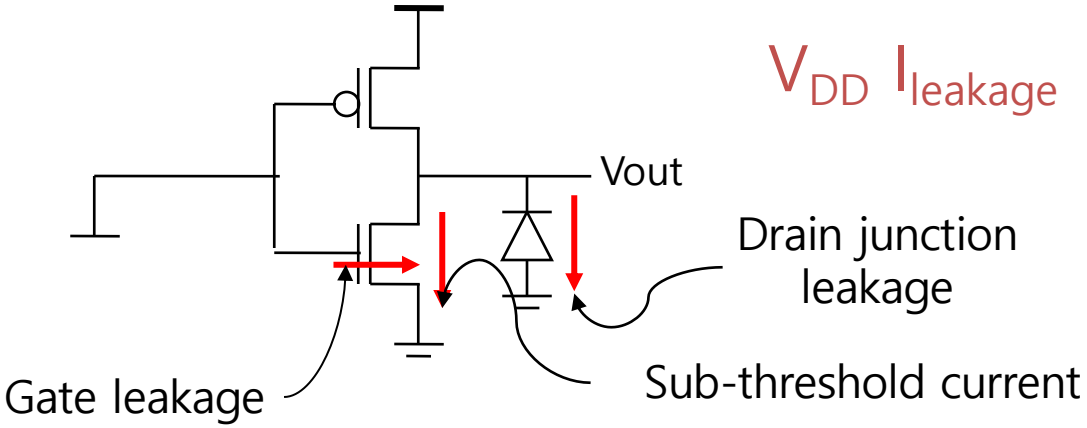
- When transistors switch, both nMOS and pMOS networks may be momentarily ON at once
- Leads to a blip of “short circuit” current.
- < 10% of dynamic power if rise/fall times are comparable for input and output





Static (Leakage) Power

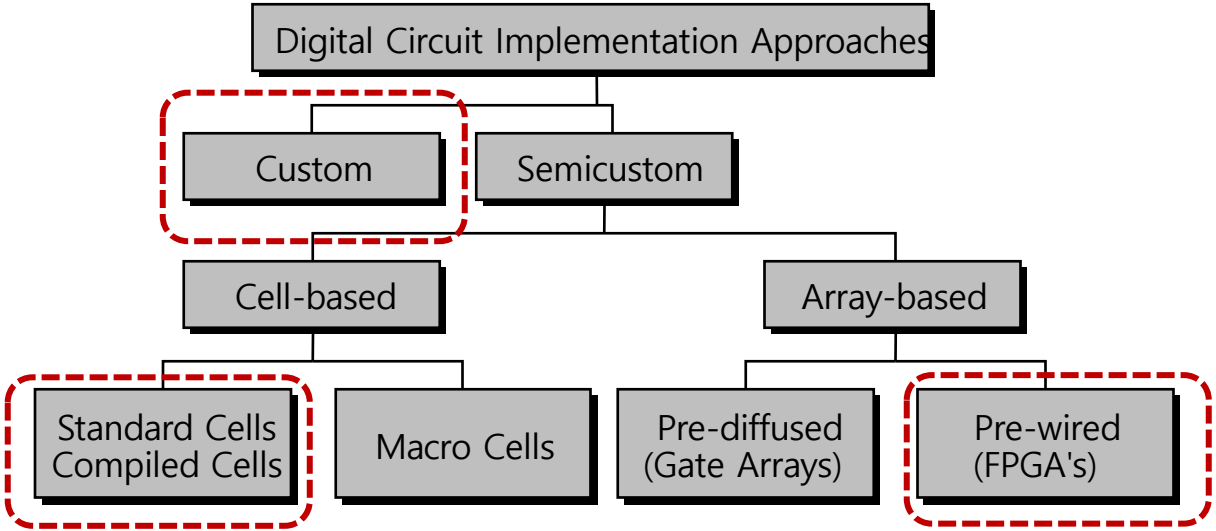
- Static power is consumed even when chip is quiescent.



Sub-threshold current is the dominant factor.
All increase exponentially with temperature!

Design Methodology

Implementation Choices

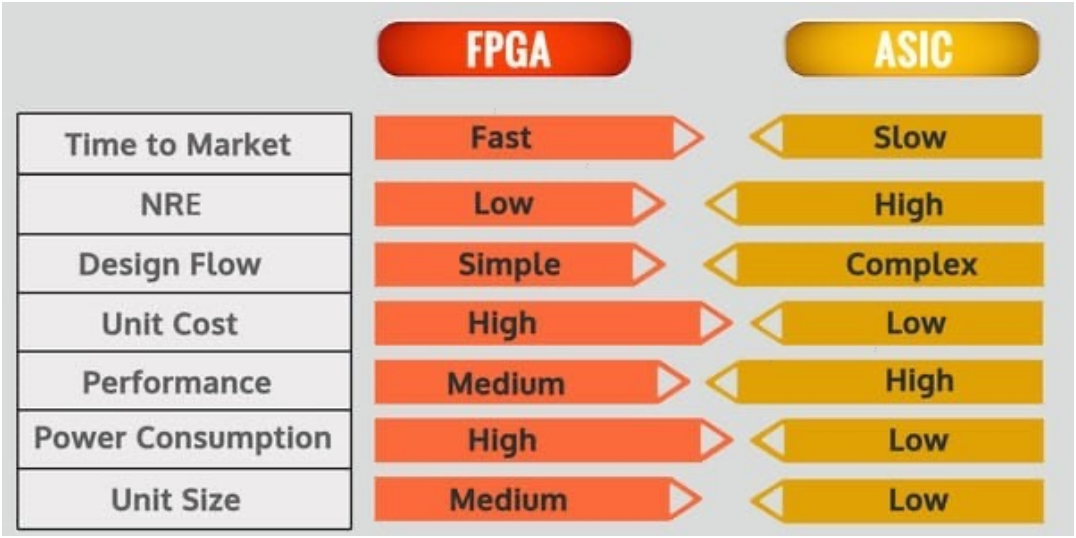


- Overview of implementation approaches for digital integrated circuit
  - Full custom / cell-based / FPGA

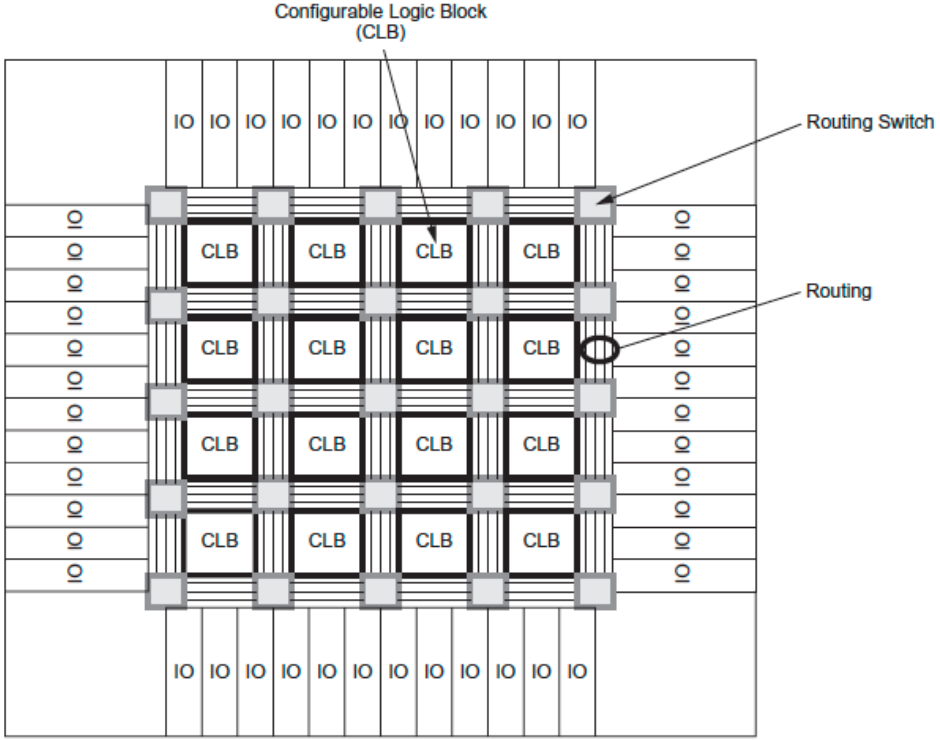
FPGA Principles

- A Field-Programmable Gate Array (FPGA) is an integrated circuit that can be configured by the user to emulate any digital circuit as long as there are enough resources

- An FPGA can be seen as an array of Configurable Logic Blocks (CLBs) connected through programmable interconnect (Switch Boxes)

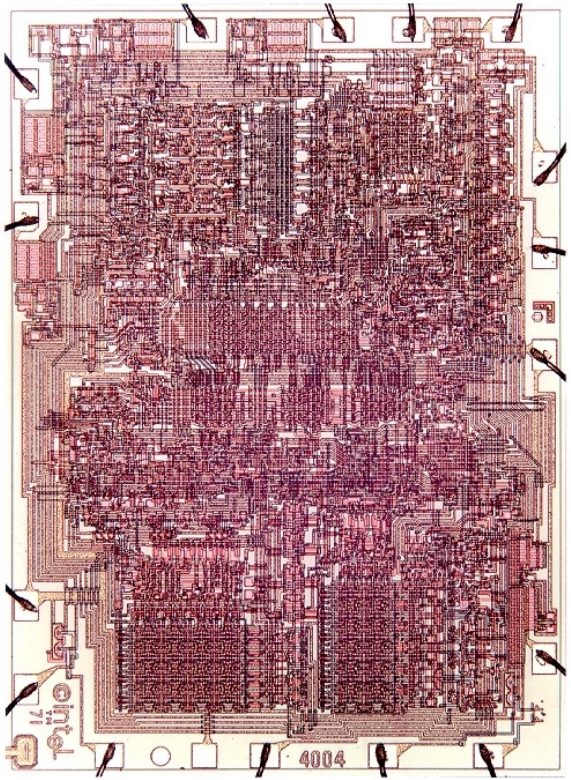


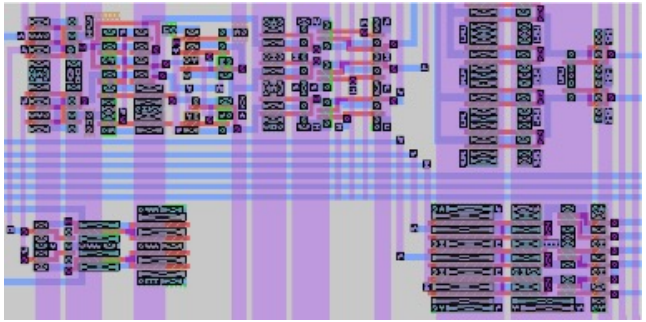
Field-Programmable Gate Arrays



Simplified FPGA floorplan

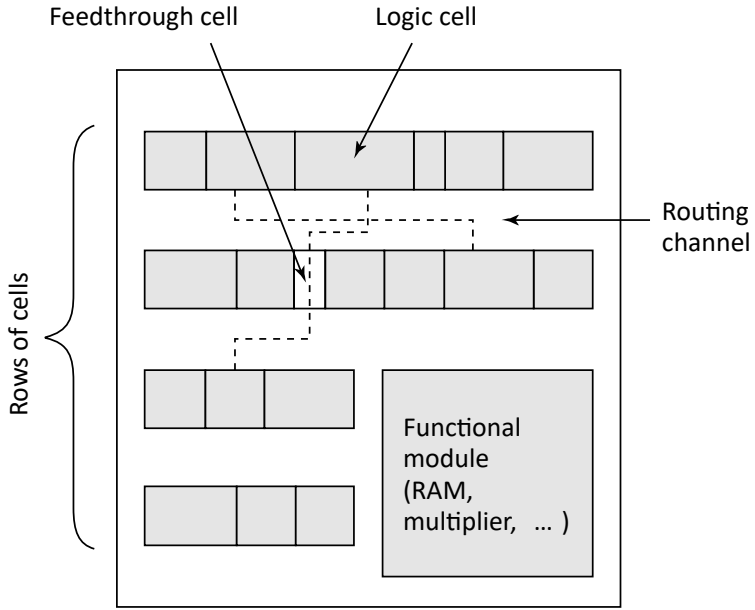
The Custom Approach



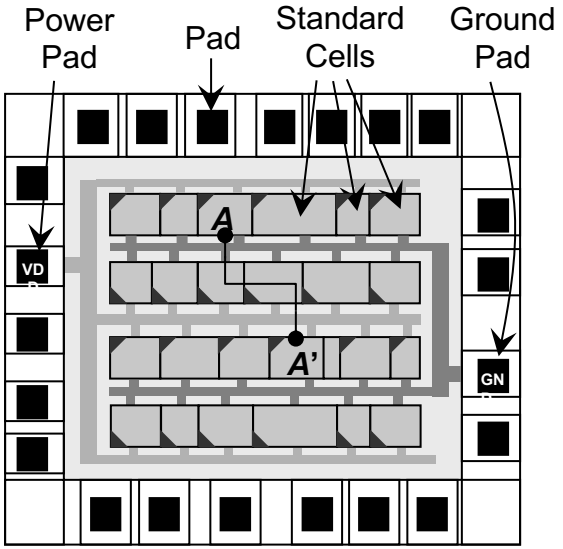


Key is that all circuits and transistors are optimized for specific context

Cell-based Design (or standard cells)

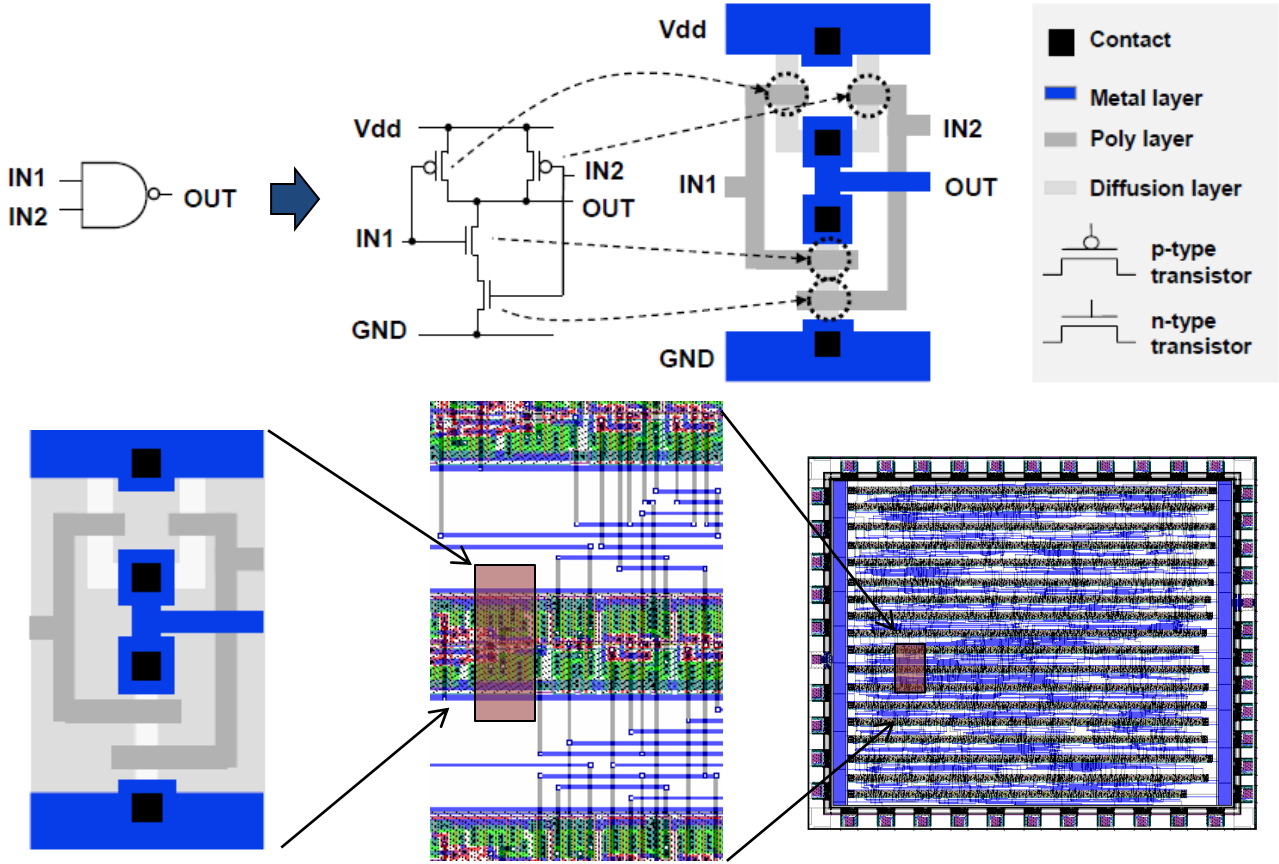


Standard cell layout using over-the-cell (OTC) routing



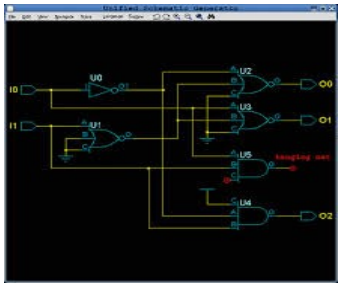
Routing channel requirements are reduced by presence of more interconnect layers

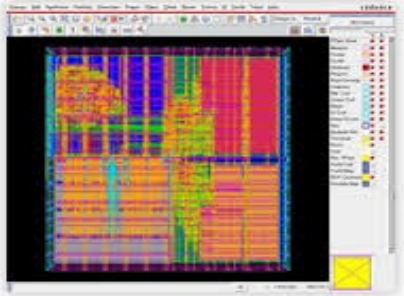
Standard-Cell Design

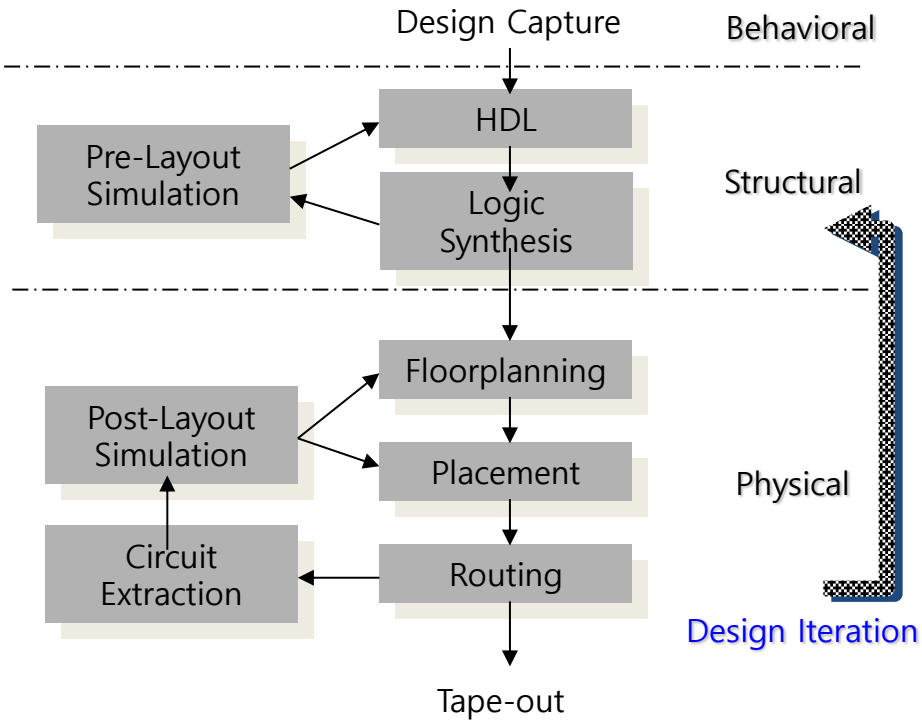


Cell-based (Semicustom) Design Flow

```
module TwoDepthDelay(
            input dataIn,
            input clk,
            input reset,
            output reg dataOut);

reg intermediate;

always @(posedge clk or posedge reset)
begin
   if(reset)
   begin
      dataOut      <= 1'b0;
      intermediate <= 1'b0;
   end
   else
   begin
      intermediate <= dataIn;
      dataOut      <= intermediate;
   end
end

endmodule
```

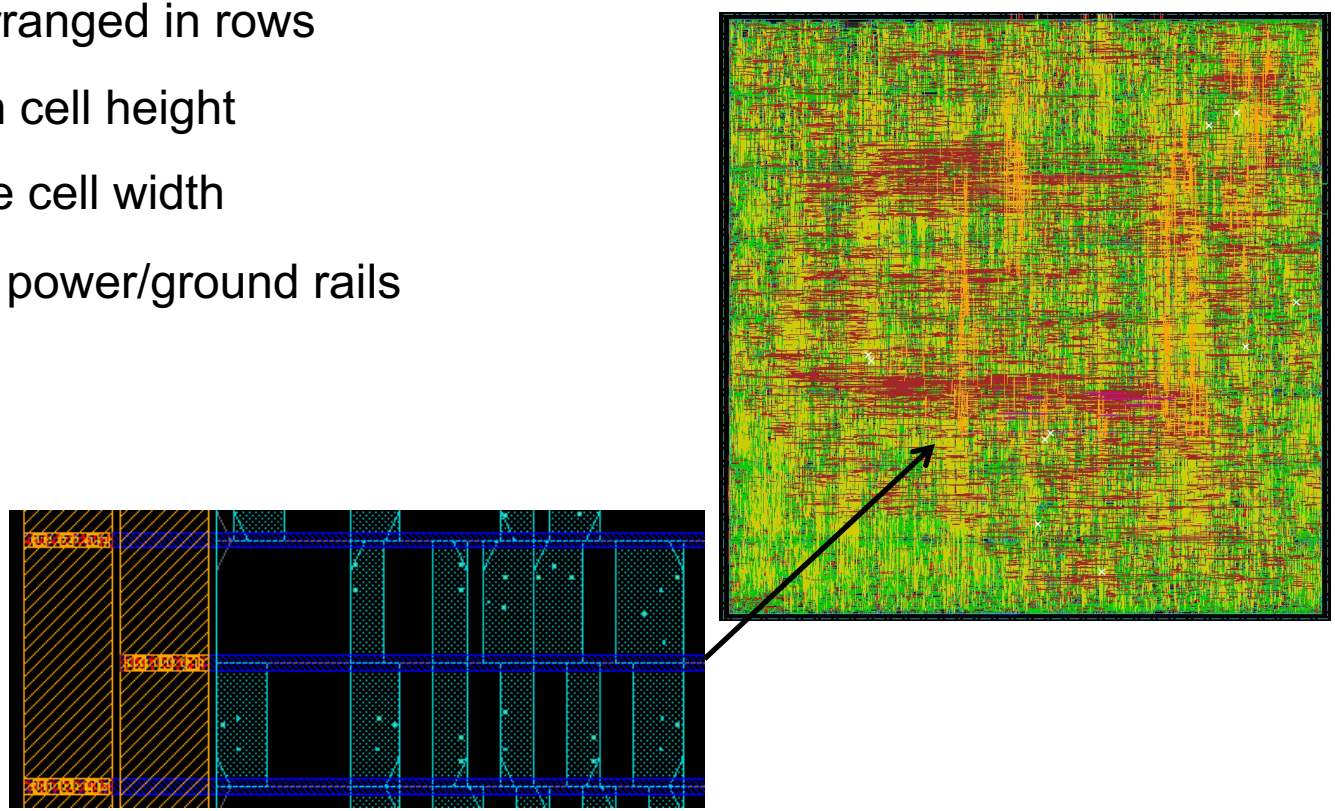




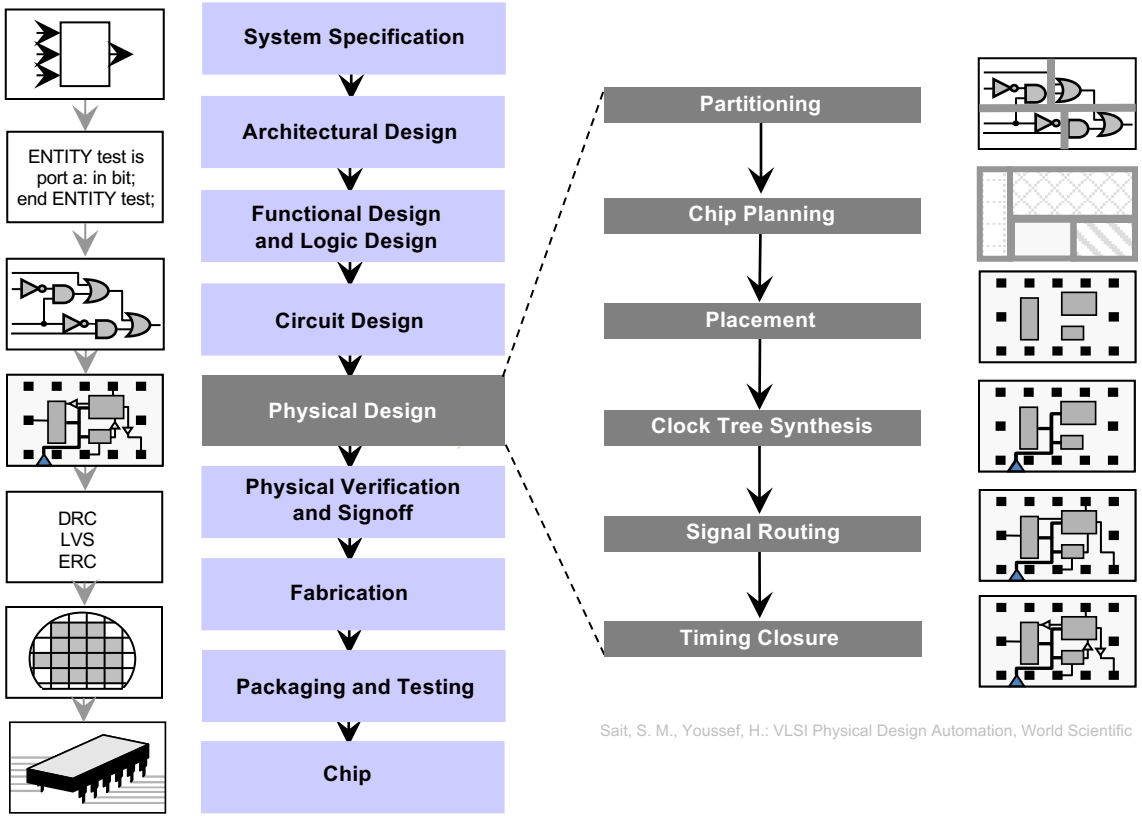


Standard-Cell Methodology

- Cells arranged in rows
- Uniform cell height
- Variable cell width
- Shared power/ground rails



Design Flow and Physical Design



Sait, S. M., Youssef, H.: VLSI Physical Design Automation, World Scientific

67

Hardware Description Languages (HDL)

- Need a description level up from logic gates
  - Work at the level of functional blocks, not logic gates
  - A functional unit could be an ALU, or could be a microprocessor
- HDL translates the specification of a hardware device into a software medium
  - Can be verified via simulation
  - Can be translated into an optimized, technology-specific, gate-level implementation (logic synthesis)
- Verilog HDL
  - Originally designed in 1984 for verification/simulation
  - 4 value logic (0, 1, x, z)
  - Provide a wide range of levels of abstraction – architectural, RTL, gate

Module Example

Verilog code Example

```
module Example1
#(
        parameter BIT_WIDTH =4
)(
        input [BIT_WIDTH -1 :0] A, B ,
        input clock,
        output [BIT_WIDTH *2 -1:0] Q
);
        // this is comment !!
        reg Q;
        wire [BIT_WIDTH :0] C;
        assign C = A + B ;

        always @ (posedge clock)
begin
                Q = A * B + C;
        end
endmodule
```

module [module_name]
module name 을 '**Example1**' 로 설정
#() 괄호 안에 Parameter 설정

A 와 B 를 4bit Input port로 선언
start, resetn, clock을 1bit Input port로 선언
Q를 8bit Output port로 선언

reg, wire 선언

Continuous Assignments

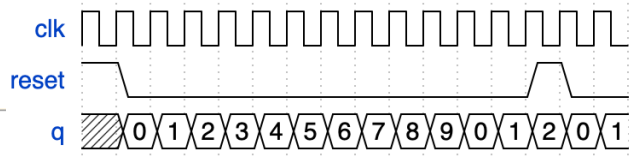
Procedural Assignments

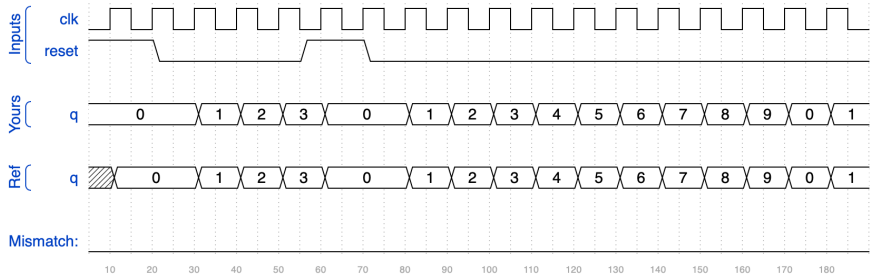
Example1 Module 끝

Verilog Practice: Counter

- Decade counter
  - https://hdlbits.01xz.net/wiki/Count10

```
1  module top_module (
2      input clk,
3      input reset,        // Synchronous active-high reset
4      output [3:0] q);
5
6      always @ (posedge clk) begin
7          if (reset) begin
8              q = 4'b0;
9          end
10         else if (q == 9) begin
11             q = 4'b0;
12         end
13         else begin
14             q = q + 1'b1;
15         end
16     end
17
18 endmodule
```

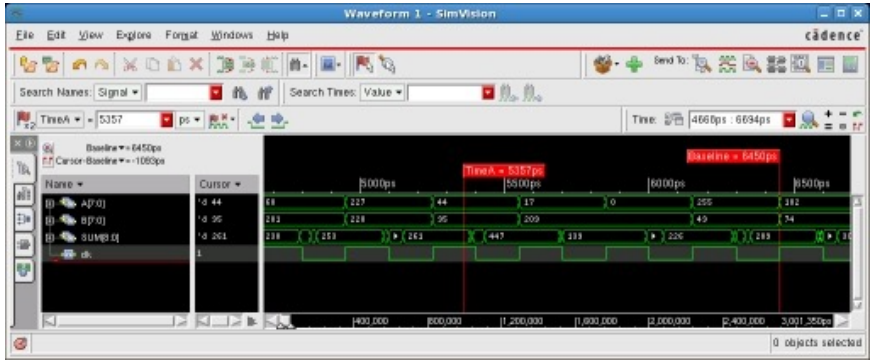




RTL Simulation

- RTL code, written in Verilog, VHDL or a combination of both, is simulated to verify functional correctness
- Testbenches apply input stimulus to the design
- Several methods are used to verify the outputs
  - Self-checking testbenches automatically verify output correctness and report mismatches
  - Results can be stored in a file and compared to previous results
  - Waveform displays can be used to interactively verify the outputs

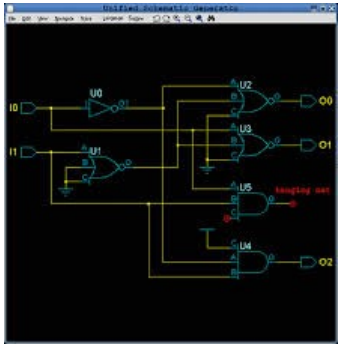
```
module TwoDepthDelay(
            input dataIn,
            input clk,
            input reset,
            output reg dataOut);

reg intermediate;

always @(posedge clk or posedge reset)
begin
   if(reset)
   begin
      dataOut      <= 1'b0;
      intermediate <= 1'b0;
   end
   else
   begin
      intermediate <= dataIn;
      dataOut      <= intermediate;
   end
end

endmodule
```



Logic Synthesis

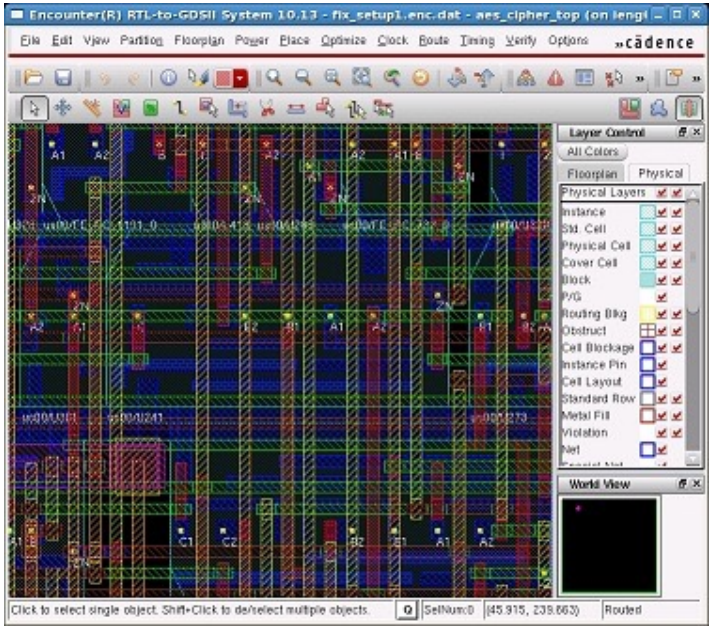
- Conversion of RTL to gate-level netlist
  - Targeted to a foundry-specific library
  - Can be performed hierarchically
- Timing-driven
  - Clock information
  - Primary input arrival times, primary output required times
- Interconnect delay (wireload model)

```
module TwoDepthDelay(
          input dataIn,
          input clk,
          input reset,
          output reg dataOut);

reg intermediate;

always @(posedge clk or posedge reset)
begin
   if(reset)
   begin
      dataOut      <= 1'b0;
      intermediate <= 1'b0;
   end
   else
   begin
      intermediate <= dataIn;
      dataOut      <= intermediate;
   end
end

endmodule
```

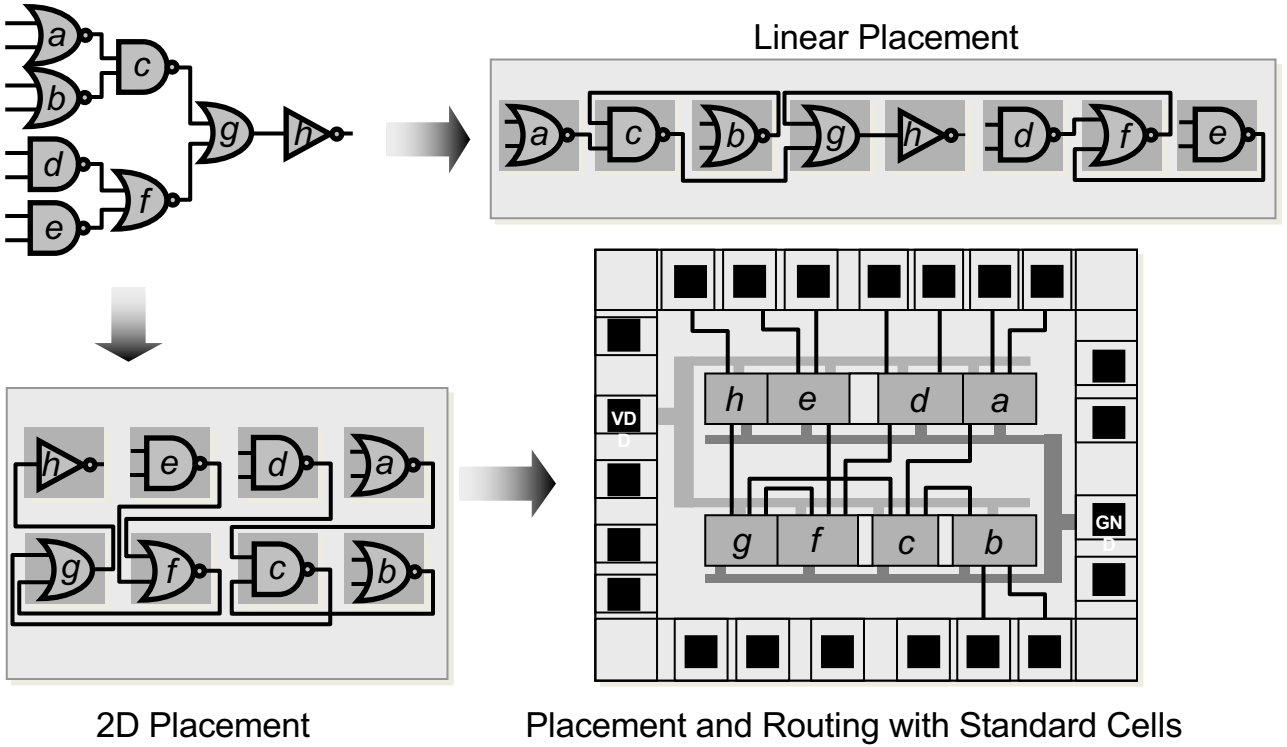


Place and Route

- Automatically place the standard cells
- Re-optimization and re-buffering
- Generate clock trees
- Detailed routing of clock lines
- Detailed routing of signal interconnects
- Re-buffering, hold time fixing
- Design rule checks



Placement Overview

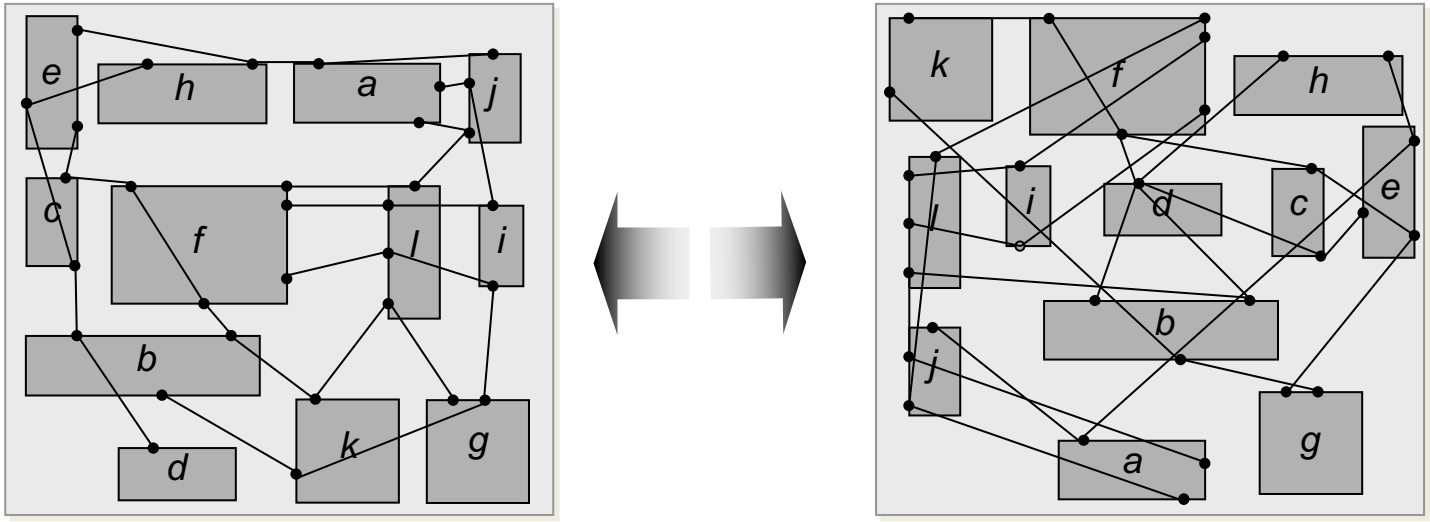


2D Placement

Placement and Routing with Standard Cells

Placement determines the locations of STD. cells or logic elements while addressing optimization objectives, e.g., minimizing the total wire length.

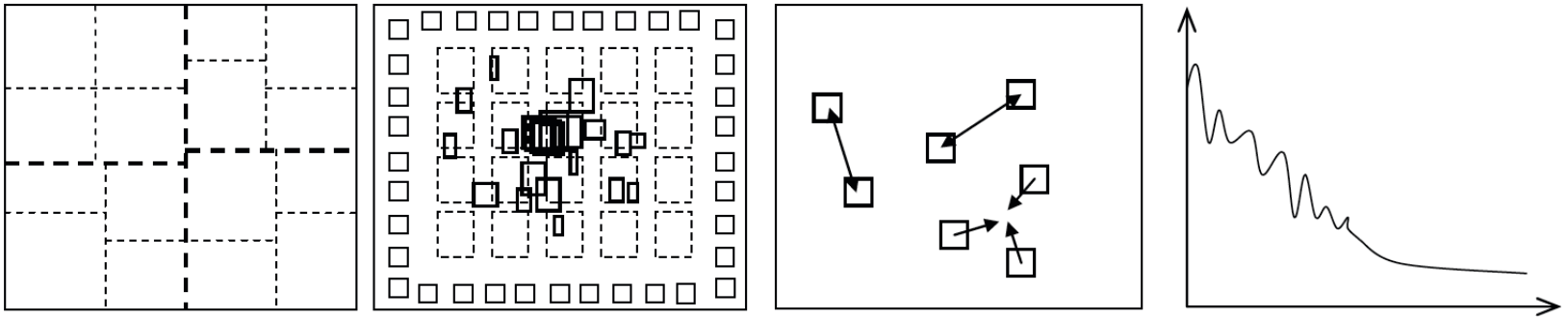
Optimization Objectives – Total Wirelength



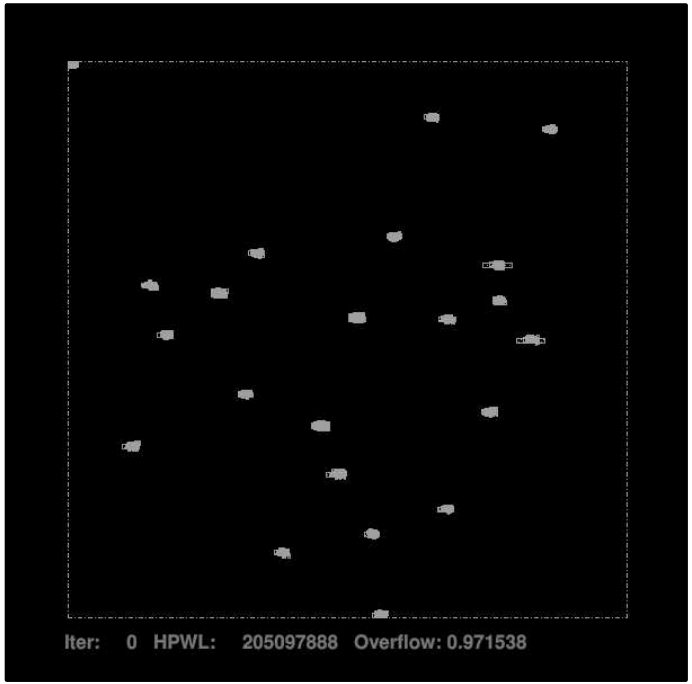
The placement of a design strongly affects the net lengths as well as the wire density.

Global Placement

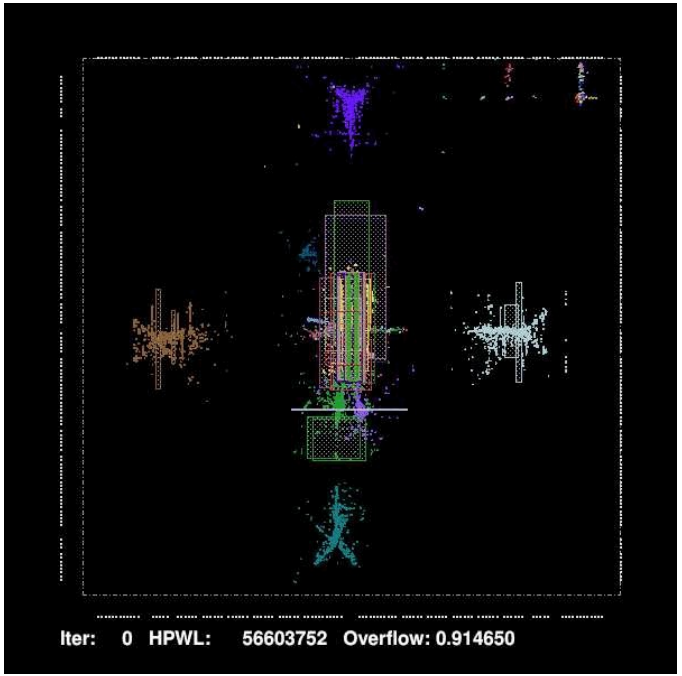
- Partitioning-based algorithms: netlist and layout are divided into smaller sub-netlists and sub-regions, according to cut-based cost function (e.g., min-cut placement)
- Analytic techniques: maximize objective function via mathematical analysis (e.g., quadratic placement, force-directed placement)
- Stochastic algorithms: optimize cost function with randomized moves (e.g., simulated annealing)



Placement Example



aes_cipher
(#std cells: ~10K)



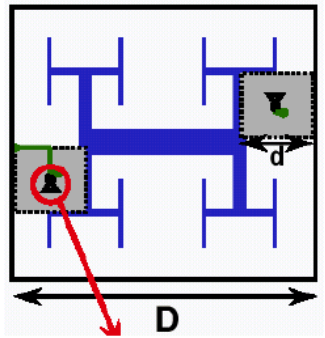
adaptec1
(#std cells: ~211K, #macros: 63)

Routing Overview

- Given placement, netlist and technology
  - determine the necessary wiring, e.g., net topologies and specific routing segments, to connect these cells
  - while respecting constraints, e.g., design rules and routing resource capacities, and
  - optimizing routing objectives, e.g., minimizing total wirelength and maximizing timing slack.

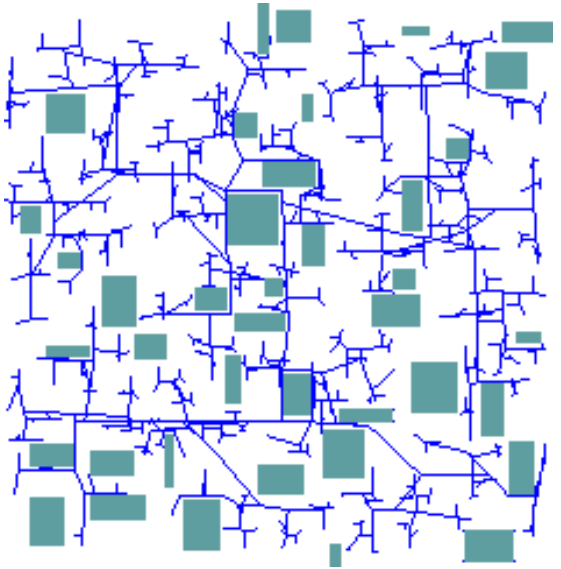
Clock Routing (Clock Tree Synthesis)

- H-tree
  - One large central driver, recursive structure to match wirelengths
  - Halve wire width at branching points to reduce reflections



courtesy of P. Zarkesh-Ha

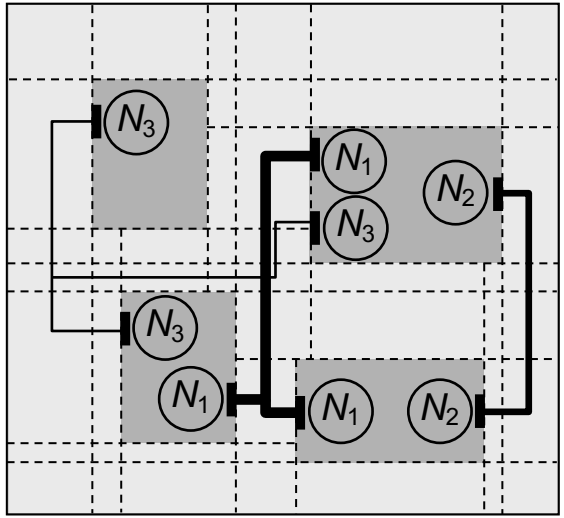
- Zero-Skew Tree
- Given a set S of sink locations and a connection topology G, construct a ZST, T(S) having minimum cost (skew).



Signal Routing: Global vs Detailed Routing

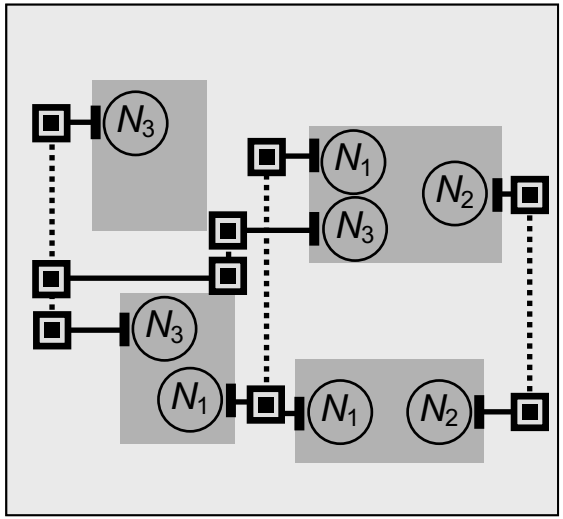
Global Routing

Coarse-grain assignment of routes to routing regions



Detailed Routing

Fine-grain assignment of routes to routing tracks



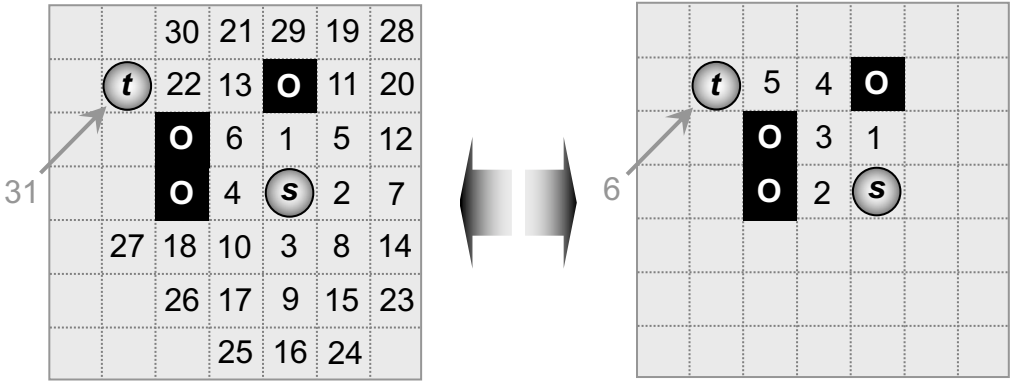
Horizontal Segment

Vertical Segment

Via

Finding Shortest Paths with A* Search

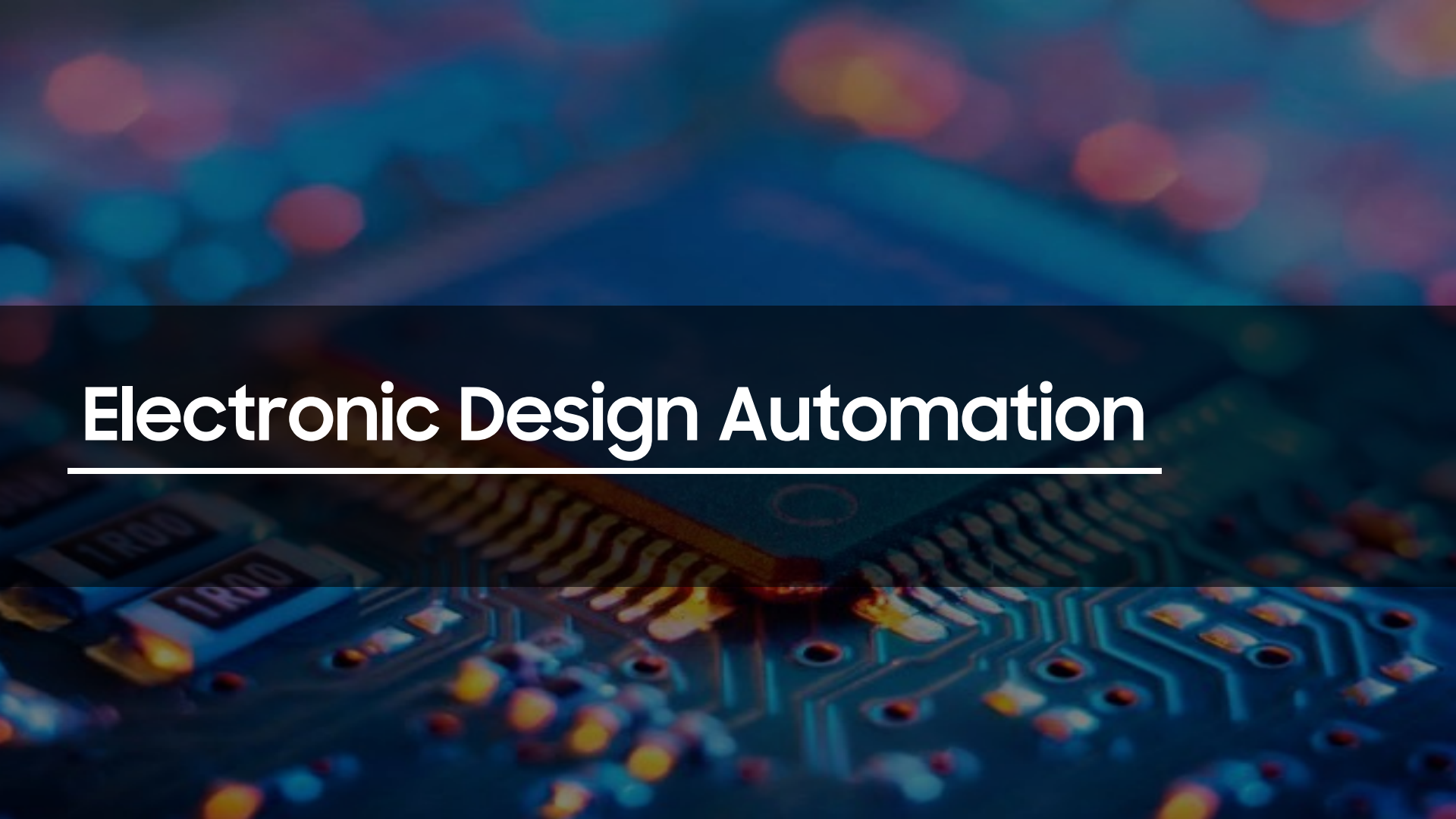
- A* search operates similarly to Dijkstra's algorithm, but extends the cost function to include an estimated distance from the current node to the target
- Expands only the most promising nodes; its best-first search strategy eliminates a large portion of the solution space



Dijkstra's algorithm
(exploring 31 nodes)

A* search
(exploring 6 nodes)

s Source t Target O Obstacle


Electronic Design Automation

EDA Market (Big 3 > 80%)

SYNOPSYS®
Silicon to Software™

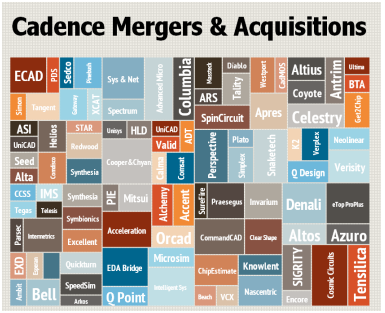
Revenue: 7B (2025)

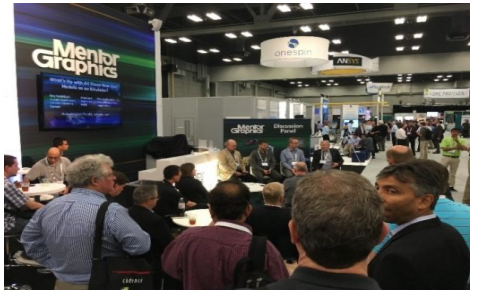
cādence®

Revenue: 5.3B (2025)

Mentor Graphics®

Revenue: ~3B (2025, Siemens EDA)

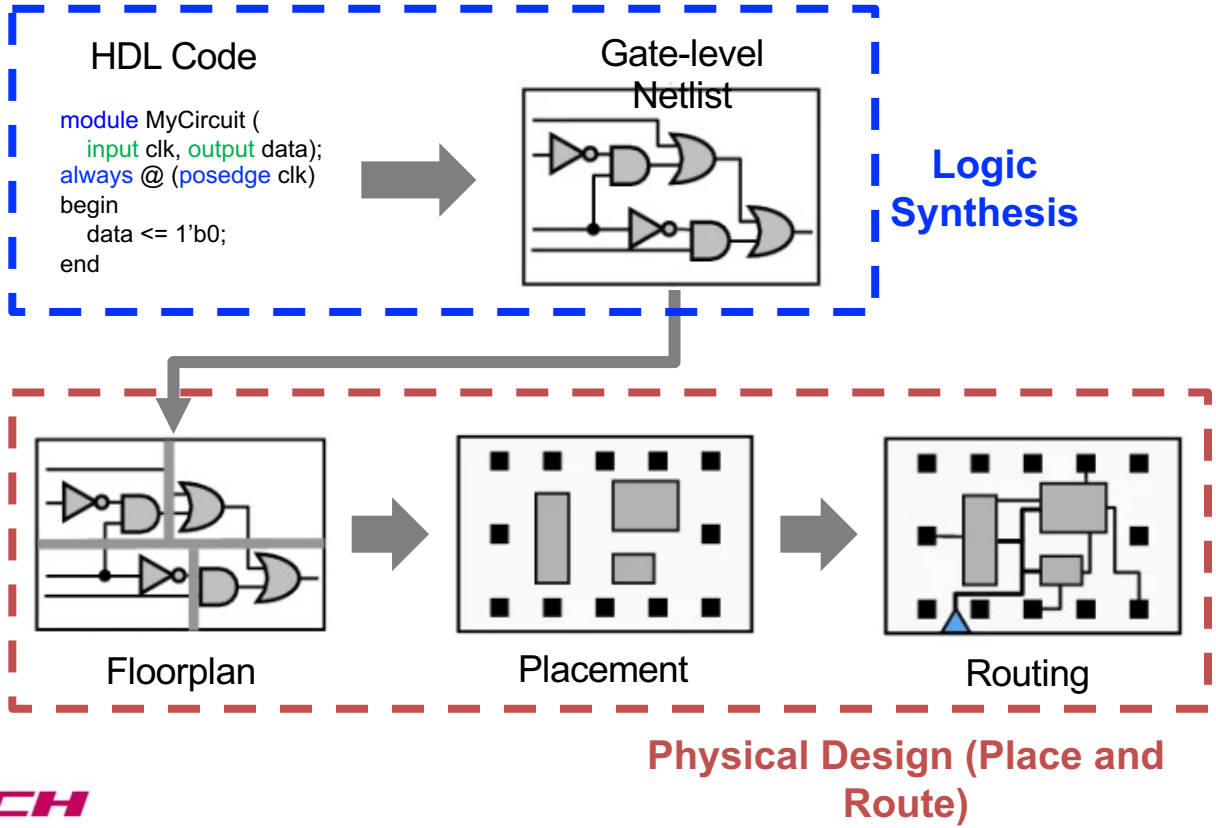




- **Current Problems with EDA SW**
  - Monopoly of Big 3 companies → slow technology development / lack of innovation
  - Reliance on the expertise or intuition of human designers → iterative and inefficient

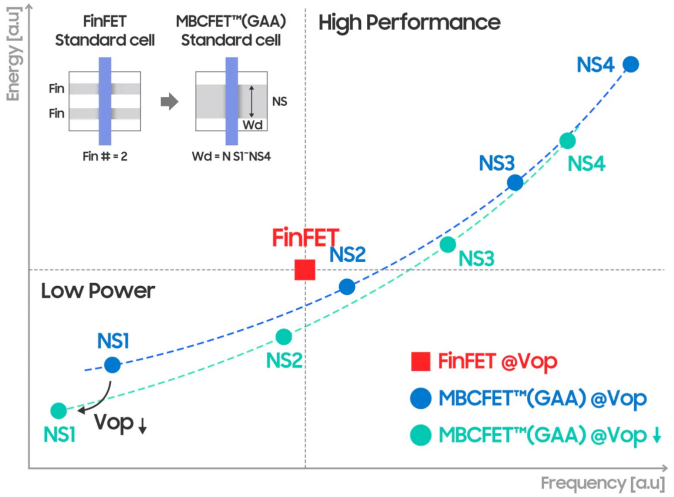
EDA – Workflow

- RTL (Register Transfer Level) to Layout



EDA Trends: 1. DTCO – Design Technology Co-Optimizatio

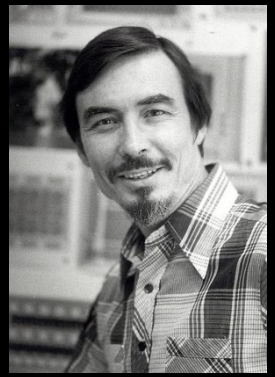
- Reaching scaling limits
- Investigate optimal solution from **both design and fabrication simultaneously**.
- Optimize various factors (OPC model, PDN architecture, PDK, cell library, etc.)
- 3nm GAA MBCFET™ of Samsung (2022 CICC)



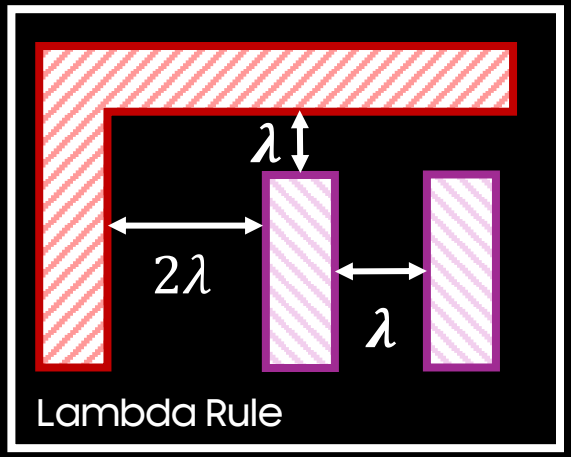
Improved PPA and flexibility of 3nm GAA with DTCO
This technology gives more PPA flexibility by adjusting channel length.

Separation of Design and Technology

- Mead – Conway Chip Design Revolution (1978)


Carver Mead


Lynn Conway



Birth of Foundry/EDA Industry

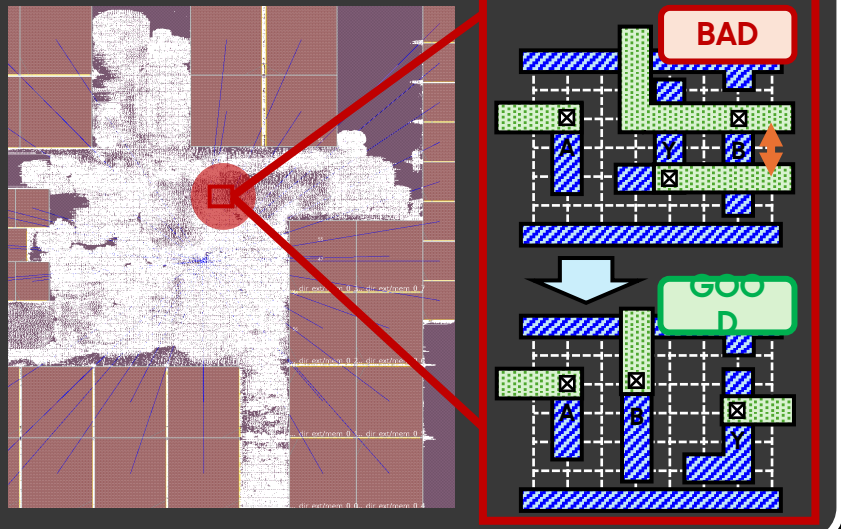
- Mentor Graphics (1981)
- Cadence (1983)
- Synopsys (1986)
- TSMC (1987)
- …

- Big success from the Separation

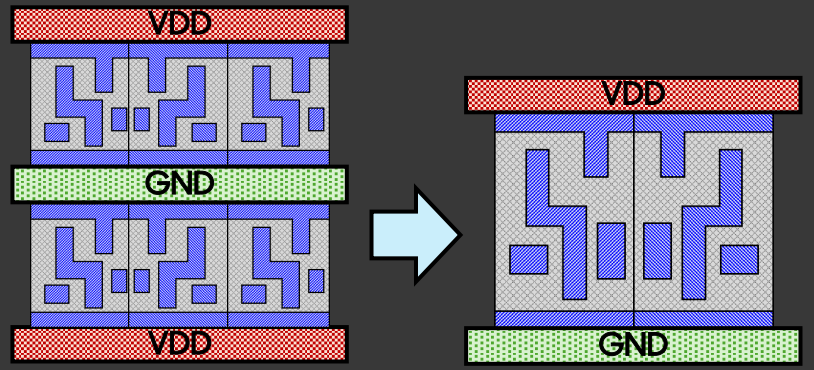
No More Separation, Co-Optimize Now

- What is DTCO? → Collaboration between design team and technology team

Design → Technology



Technology → Design

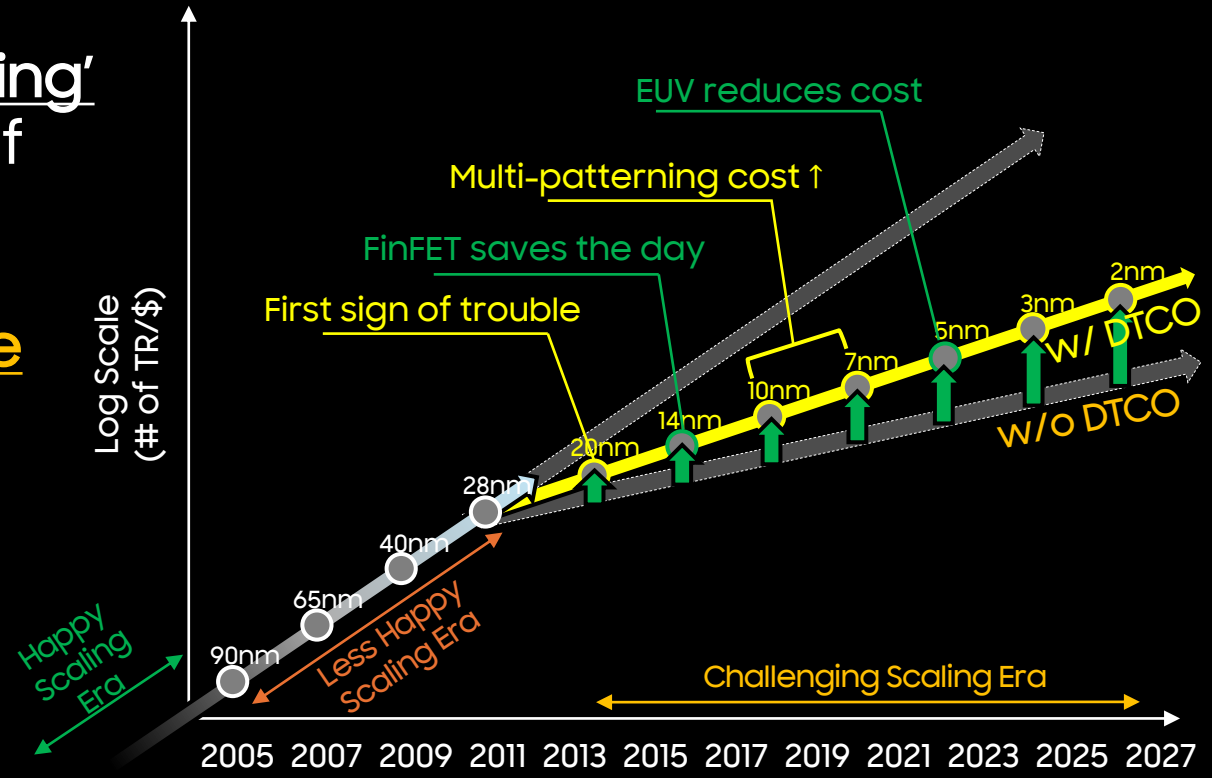


Small Cell
(Speed ↓, Power ↓)

Tall Cell
(Speed ↑, Power ↑)

No Scaling without DTCO

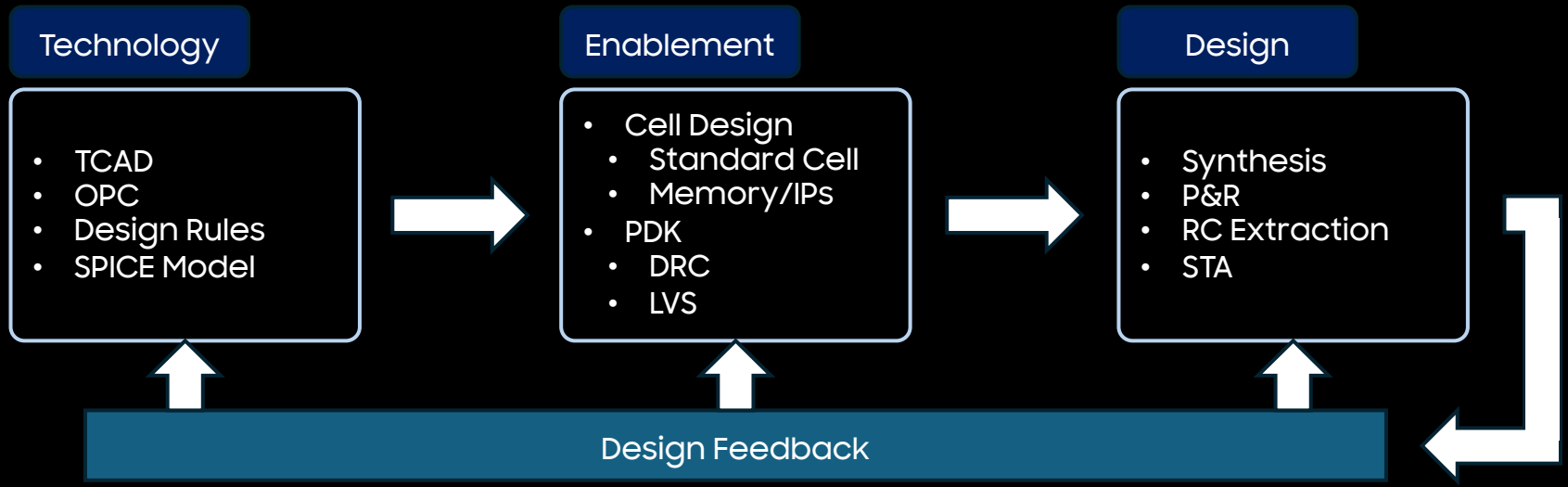
- An era of 'happy scaling' – driven by the Law of Gordon Moore
- Last 15 years, no more happy scaling
- Scaling has been complemented by DTCO in recent years



[15] IMEC Magazine, July 2019

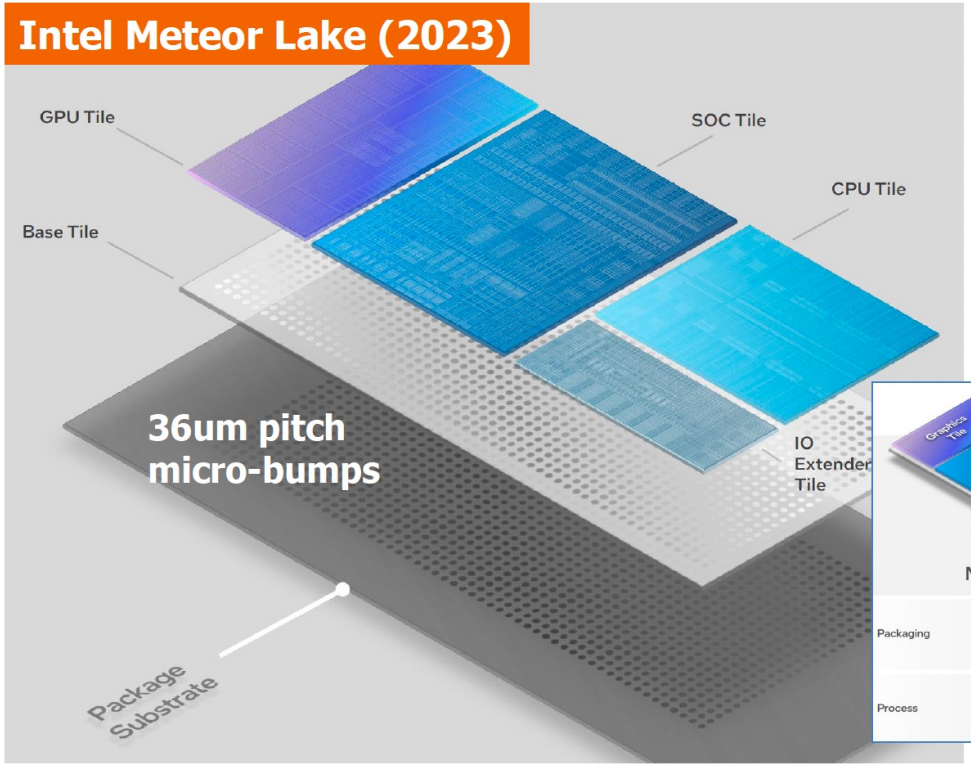
DTCO Loop Acceleration

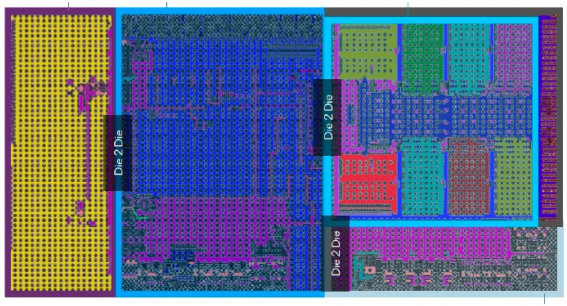
- Advanced nodes require cell- and block-level validation
- Full design-level DTCO iterations take weeks to months
- Acceleration must span Technology → Enablement → Design



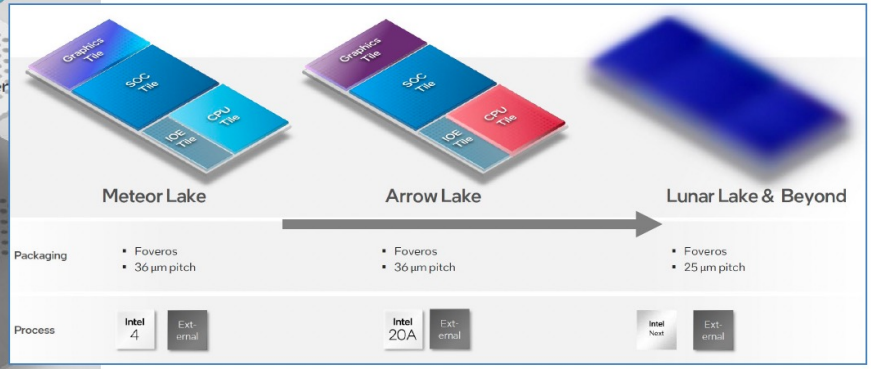
[A2] C.-K. Cheng et al., "PROBE2.0: A Systematic Framework for Routability Assessment From Technology to Design in Advanced Nodes," TCAD, 2021

EDA Trends: 2. 3D Integration





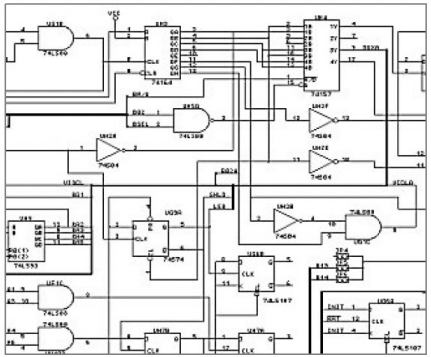
"We need new EDA tools." V. Rajan

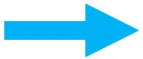


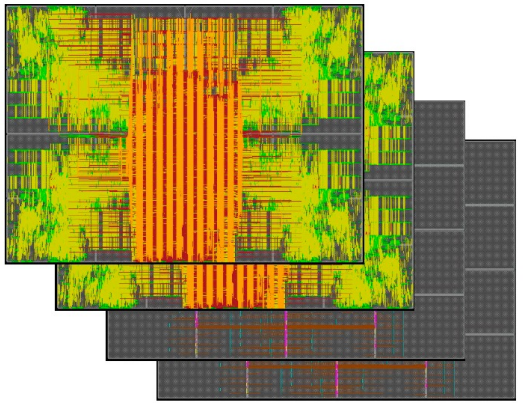
"This is our future." B. Brennan

EDA for 3D IC

3D IC Placement and Routing: MUST (not optional)







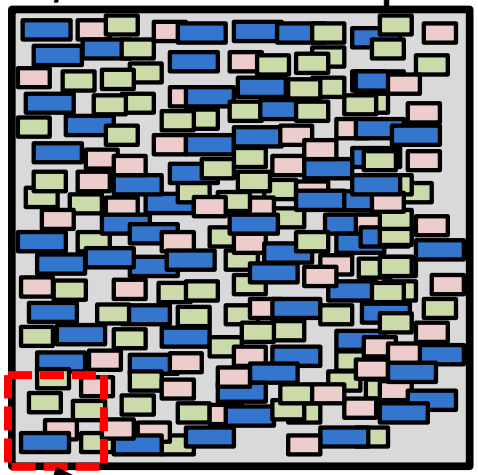
Digital circuit netlist

3D IC GDS

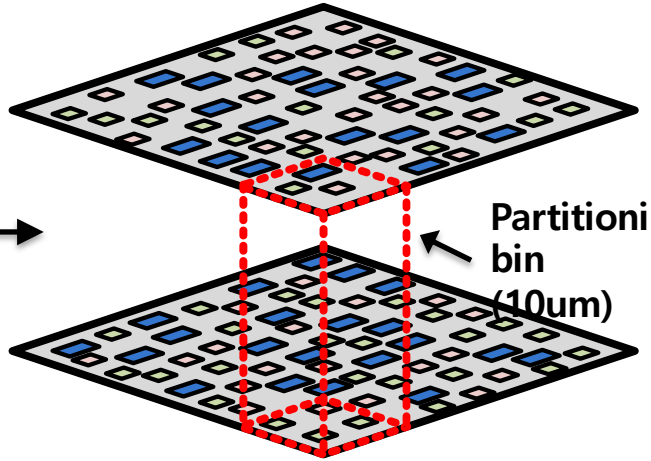
Can NOT do this manually; need algorithms to AUTOMATE

Using a 2D Placer for 3D Placement

First, make the 3D footprint 50% of 2D



In a 2D placer, simply double the placement capacity of each global bin (for two-tier) .

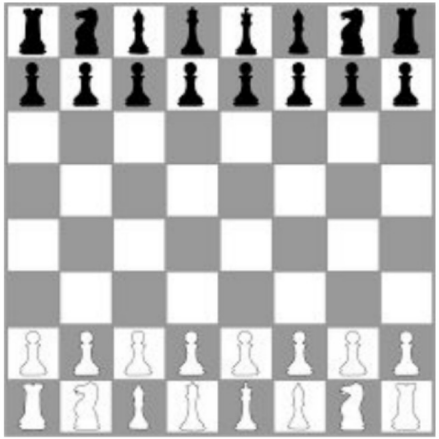


Partition the design, maintaining local area balance within each partitioning bin
"Placement-driven Partitioning"

Source: Sungkyu Lim, "Placement-Driven Partitioning for Congestion Mitigation in Monolithic 3D IC Designs"

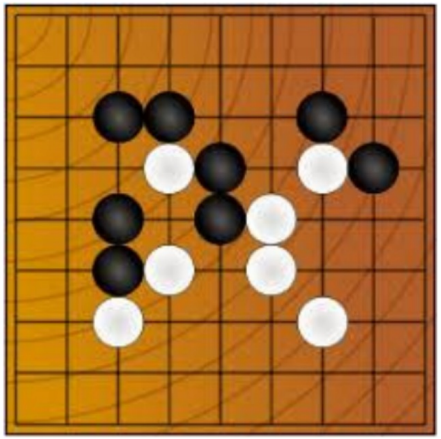
EDA Trends: 3. AI in Design Automation

Chess



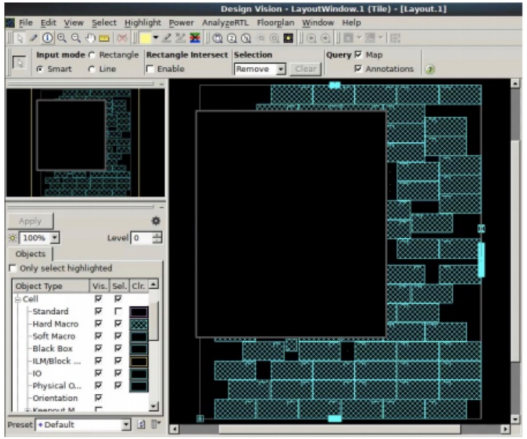
Number of states ~ 10^{123}

Go



Number of states ~ 10^{360}

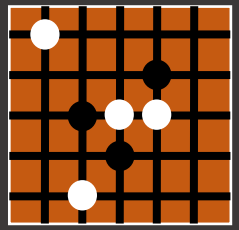
Chip Placement



Number of states ~ 10^{9000}

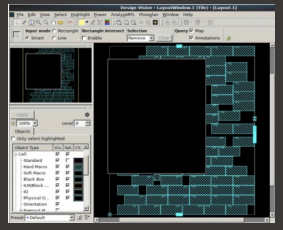
Chip Placement: Harder than Go, Tailor-made for AI

Go

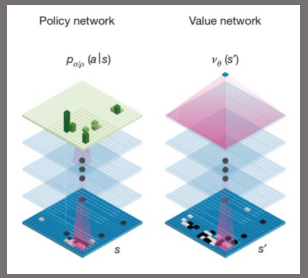


Number of State ~ 10^{360}

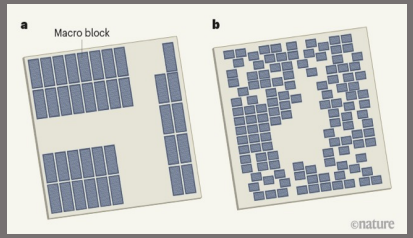
Chip placement



Number of States ~ 10^{90000}



Google - AlphaGo



Google - RL placement

Reinforcement Learning

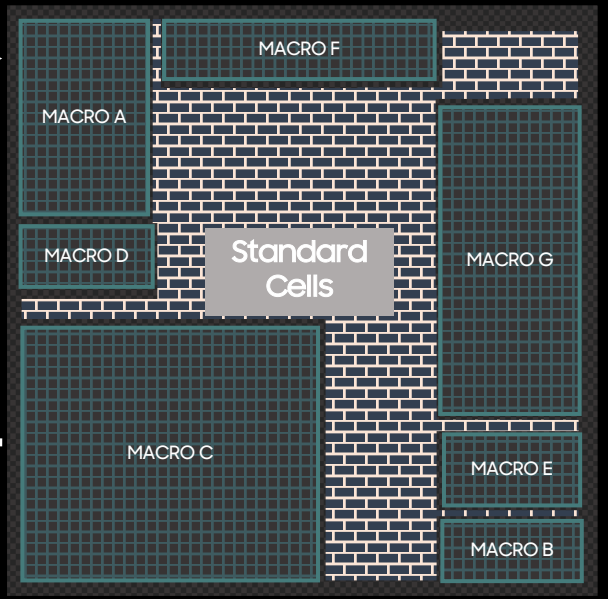
RL agent

Place macro cells
Place standard cells

Agent Update

Placement Score

Wirelength
Congestion



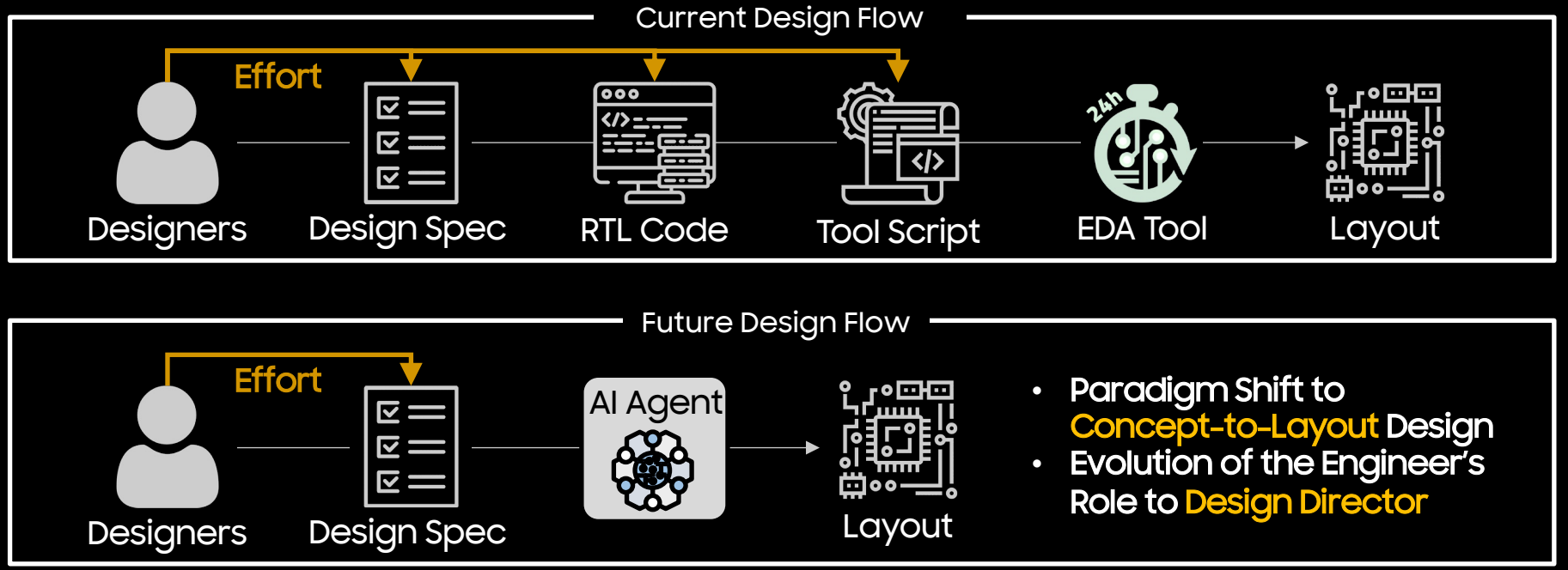
Google TPU floorplans: AI-generated in hours, better than human results

Exploration

[18] D. Silver et al., "Mastering the game of Go with deep neural networks and tree search," 2016 Nature
[19] A. Mirhoseini et al., "A graph placement methodology for fast chip design," 2021 Nature

Future of EDA with Agentic AI

- AI will autonomously drive entire design process, from high-level specifications to GDS




THANK YOU